

12. 云原生

冯奕
2026



南京大學
NANJING UNIVERSITY

目录

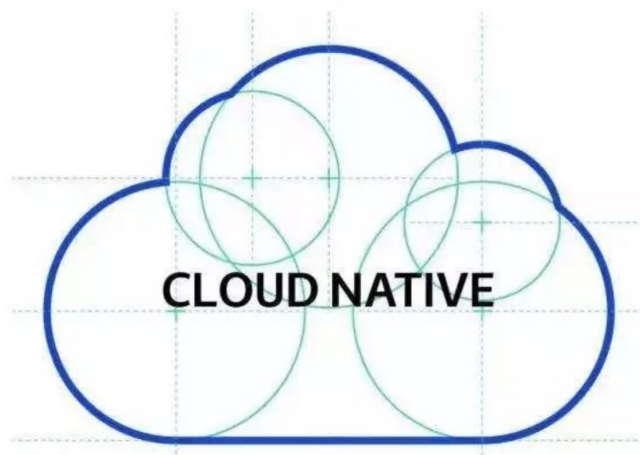
- 云原生
- 移动云云原生
- 移动云云原生全景图
- 云原生产品家族
 - 容器
 - 开发者服务
 - 数据库
 - 消息队列
 - 大数据
 - 人工智能
- 应用场景
 - 企业级云原生数据服务场景
 - 迁移上云场景
 - 服务治理场景
 - 多云混合编排场景
- 解决方案
 - 政务云原生解决方案
 - 医疗云原生解决方案



0 云原生简介

云原生是一种**构建和运行应用程序的理念和方法**，它充分利用云计算模型的优势，使应用能够在动态、弹性的现代云环境中最大化发挥其业务价值与技术潜力。

- 主旨是构建运行在云端的应用程序，致力于使应用程序能够最大限度地利用云计算技术特性的优势，提供更加优质的应用服务。
- 充分利用了云计算的优势，重点关注如何在云计算交付模式下创建和部署应用程序



- 不是单一技术，而是体系集合
- 天生适应云环境的多维价值
- 以应用为中心的思维范式转变

0 云原生关键词



分布式架构部署

将单体应用拆分为多个独立微服务，灵活部署在不同网络节点，降低系统耦合度。



极致弹性资源调度

系统可根据实时业务负载动态自动扩缩容计算资源，确保性能同时最大化控制成本。



全链路自动化运维

深度整合CI/CD流水线，实现从代码构建、容器镜像打包到生产环境部署的全流程高度自动化。



故障智能自愈保障

具备强大的健康探针机制，能自动检测组件或服务故障并触发重启或迁移，无需人工干预。



全栈深度可观测性

通过日志聚合、指标监控和链路追踪构建统一视图，实现对系统运行状态的无死角实时感知。



不可变基础设施范式

基础设施环境通过代码进行版本化定义，任何修改都通过重新构建和部署镜像来实现，杜绝环境差异。

0 云原生历程



物理机时代

2000年以前

单服务器单应用部署模式，资源隔离性强但利用率极低，软件环境配置繁琐，系统部署和扩展周期漫长。



虚拟机时代

2000 - 2013年

通过Hypervisor层实现一台物理机运行多套独立操作系统，显著提升了资源利用率，但虚拟机镜像体积庞大且启动速度较慢。



容器化时代

2013 - 2017年

Docker技术横空出世，基于Namespace和Cgroup实现进程级隔离，镜像轻量秒级启动，确立了“一次编写，到处运行”的交付标准。



云原生全面爆发

2017年至今

Kubernetes成为编排领域的事实标准，可观测性、服务网格等生态日益完善，推动应用架构向极致弹性和可韧性方向深度演进。

0 云原生价值

01 业务价值



加速业务敏捷创新迭代

开发运维一体化流水线赋能，产品发布周期从传统月/周级大幅缩短至天/小时级。



极致弹性从容应对流量洪峰

毫秒级自动扩缩容机制，轻松应对双十一、秒杀等极端业务场景，保障服务连续性。



精细化运营显著降低资源成本

通过智能调度提升整体资源利用率30%-50%，实现按量付费，最大化IT投入产出比。

02 技术底座



全链路系统稳定性深度保障

构建应用自愈能力与故障自动隔离机制，支持业务无感灰度发布，从架构层面消除单点隐患。



标准化赋能全场景开发提效

统一交付标准与运行环境一致性，消除“在我电脑上能运行”的难题，提升代码交付质量与效率。

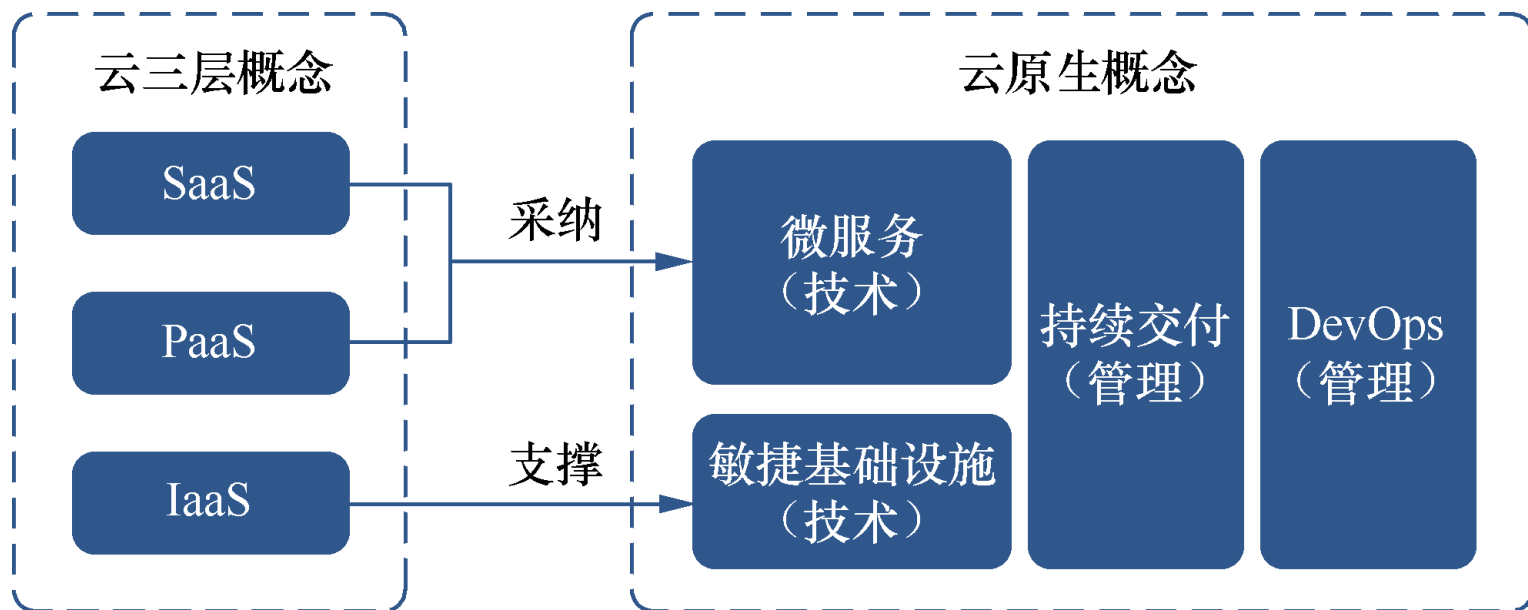


AIOps 驱动运维模式自动化变革

大幅减少重复性人工操作，将运维人员从繁琐工作中解放，有效降低人为失误引发的系统故障风险。

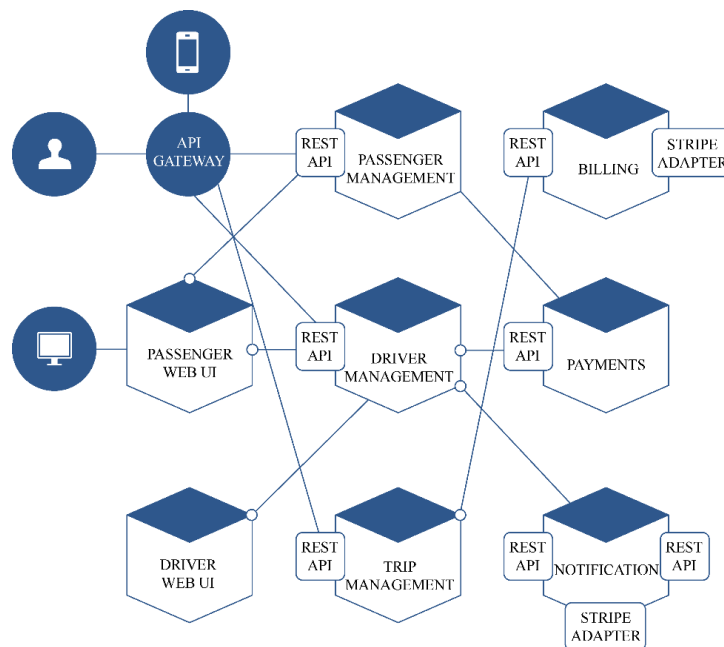
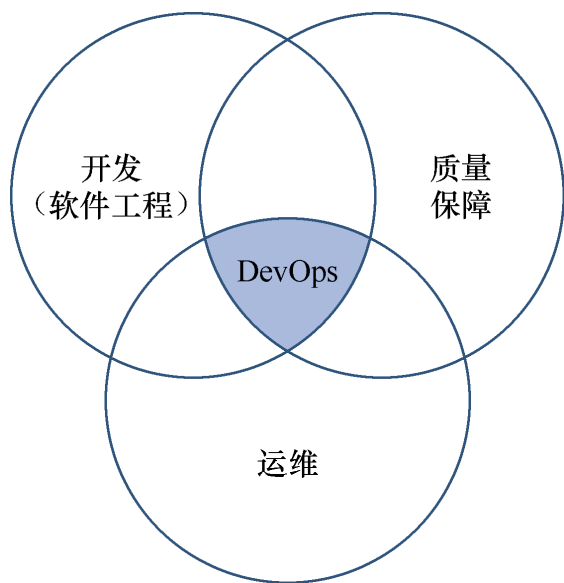
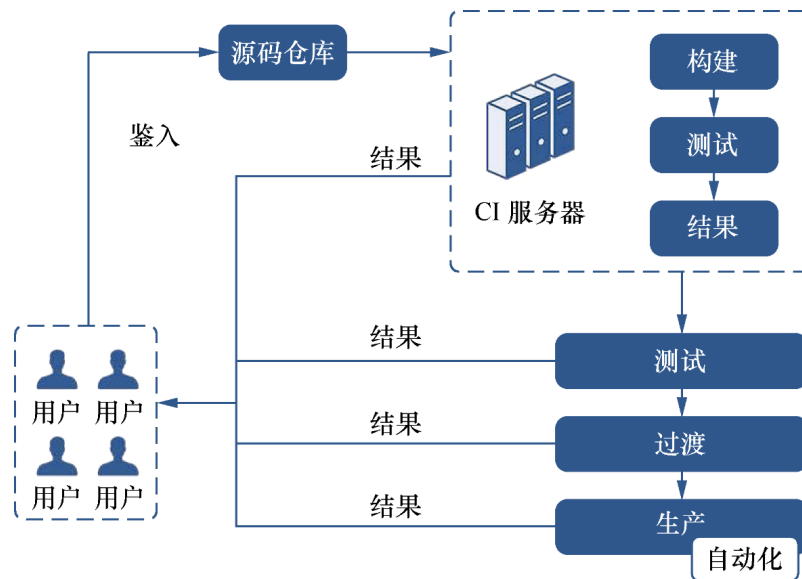
0 云原生内容

- 云原生是面向“云”设计的应用，因此技术部分依赖于传统云计算的三层概念。
- 应用基于云服务进行架构设计，对技术人员的要求更高。
- 借助云服务提供的能力实现更优雅的设计



0 云原生内容

- 敏捷基础设施
- 持续交付
- DevOps
- 微服务



0 云原生内容

敏捷基础设施

- 业务代码承载产品诉求，版本管控支撑业务快速迭代。同理，云原生开发模式下，基础资源也需依托代码实现自动化调度、变更留痕，并保障全环境统一。
- 运维人员将服务器部署、模板管理、版本更新及基础设施定义等工作全部代码化、自动化落地，摒弃传统手动安装、服务器克隆等人工运维方式。
- 与此同时，基础设施的覆盖范畴也进一步拓展，除物理主机外，还涵盖机柜、网络交换机，以及同城、异地多机房等各类软硬件与跨区域资源

0 云原生内容

持续交付

- 面对频繁变化的业务诉求，持续交付依托快速迭代机制，打造可随时上线发布的软件交付能力，是一套完整的工程实践体系。该体系包含持续集成、持续部署、持续发布等核心环节，打通从需求梳理、设计开发、自动化测试，到代码安全、高效部署至生产环境的全流程。
- 持续集成要求开发人员每一次代码提交，都会自动触发项目构建与自动化测试，以此校验应用与服务的运行效果是否符合预期。
- 在持续集成的基础上，持续交付可将稳定版本推送至预发布环境，使产品达到可上线投产的标准。
- 持续部署则进一步实现全流程自动化，代码变更完成后可自动分发至对应测试环境

DevOps

- DevOps从字面上理解只是Dev（开发人员）+ Ops（运维人员），实际，它是一组过程、方法与系统的统称。
- （1）组织架构、企业文化与理念等，需要自上而下设计，用于促进开发部门、运维部门和质量保障部门之间的沟通、协作与整合。
- （2）自动化是指所有的操作都不需要人工参与，全部依赖系统自动完成。
- （3）DevOps的出现是由于软件行业日益清晰地认识到，为了按时交付软件产品和服务，开发部门和运维部门必须紧密合作。

微服务

随着企业的业务发展，传统业务架构面临着很多问题。

- 单体架构在需求越来越多的时候无法满足其变更要求，开发人员对大量代码的变更会越来越困难，同时也无法很好地评估风险，所以迭代速度慢。
- 系统经常会因为某处业务的瓶颈导致整个业务瘫痪，架构无法扩展，木桶效应严重，无法满足业务的可用性要求。
- 整体组织效率低下，无法很好地利用资源，存在大量的浪费。

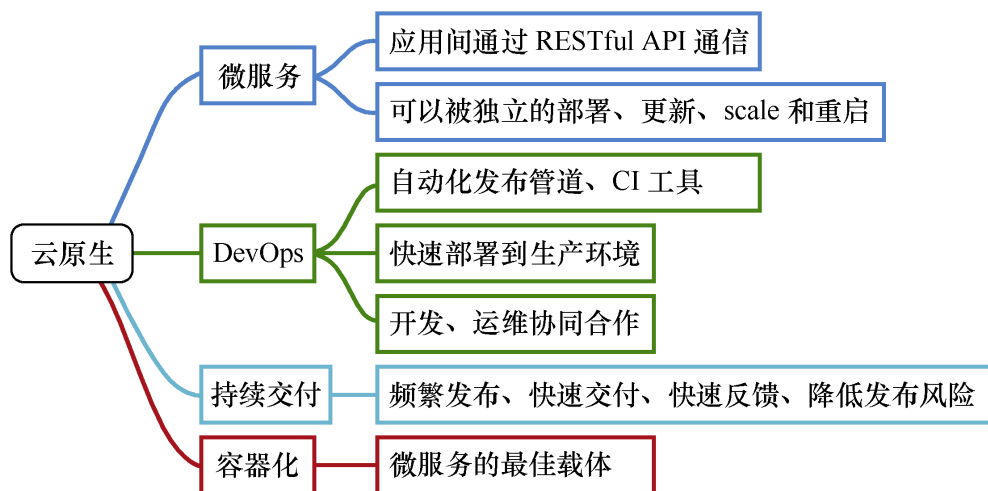
微服务是一种架构风格，也是一种服务。

- 微服务的颗粒比较小，一个大型复杂软件应用由多个微服务组成，例如Netflix目前由500多个微服务组成；
- 它采用UNIX设计的哲学——每种服务只做一件事，是一种松耦合的、能够被独立开发和部署的无状态化服务（独立扩展、升级和可替换）。

0 云原生应用的技术手段

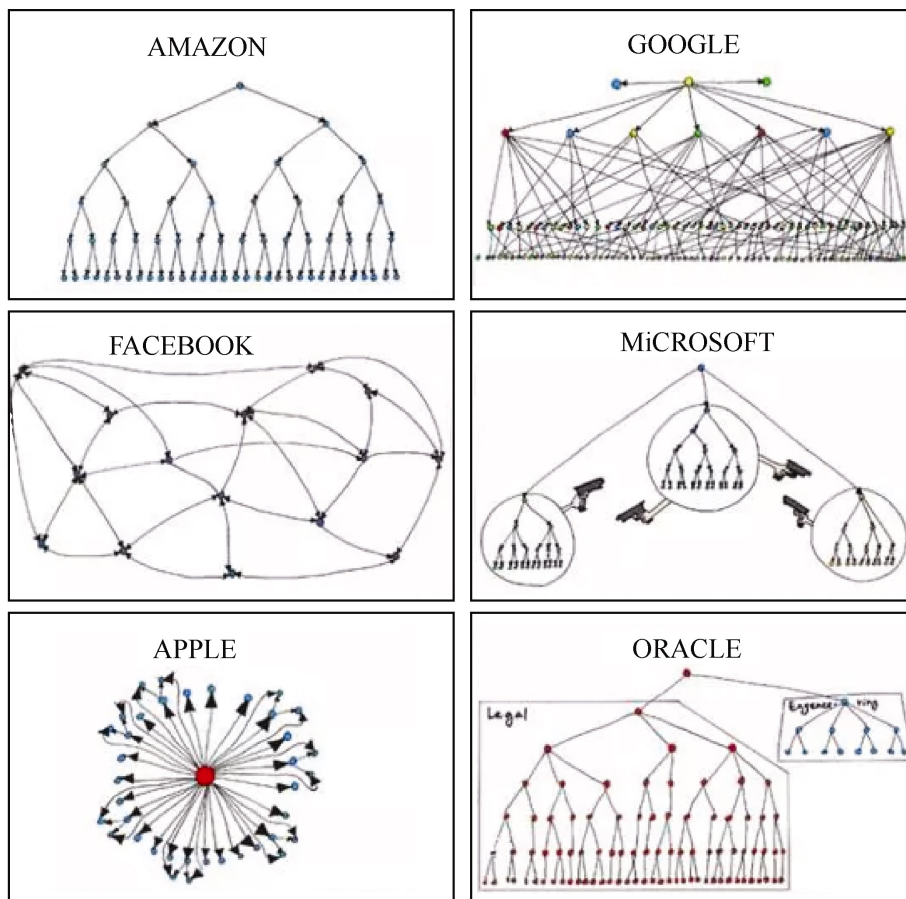
云原生四要素：DevOps、持续交付、微服务、容器

- 从宏观概念上讲，云原生是不同思想的集合，集目前各种热门技术之大成。
- 在实际的云原生开发过程中，团队需要一个构建和运行云原生应用程序的平台，这个平台需要具有高度自动化和集成化的特点。从具体的技术手段来说，它会涉及微服务、DevOps、持续集成（Continuous Integration, CI）与持续交付（Continuous Delivery, CD）、容器等技术。



0 云原生应用的技术手段-微服务

微服务领域有一个著名的“康威定律”：设计系统的组织、最终产生的设计等同于组织之内、之间的沟通结构。这意味着设计系统的企业生产的设计等同于企业内的沟通结构。



0 云原生应用的技术手段-DevOps

DevOps技术通过自动化软件交付和架构变更的流程，使得构建、测试、发布软件能够更加地快捷、频繁和可靠

可以把DevOps看作开发（软件工程）、技术运营和质量保障三者的交集。传统的软件组织将开发、IT运营和质量保障（设为各自独立的部门，在这种环境下如何采用新的开发方法（例如敏捷软件开发）是一个重要的课题。

DevOps考虑的还不只是软件部署，它是一套针对这几个部门间沟通与协作问题的流程和方法。需要频繁交付的企业可能更需要了解DevOps。

0 云原生应用的技术手段-DevOps

以下几方面因素可能促使一个组织引入DevOps：

- 使用敏捷或其他软件开发过程与方法；
- 业务负责人要求加快产品交付的速度；
- 虚拟化和云计算基础设施（可能来自内部或外部供应商）日益普遍；
- 数据中心自动化技术和配置管理工具的普及。

0 云原生应用的技术手段-DevOps

DevOps的落地实现需要通过一套集成的工具链，具体包括以下目标：

- 开发、交付和运维工具之间的实时协作；
- 实现从需求获取和需求评审到设计和代码分析的持续规划；
- 落实测试策略以实施持续测试；
- 当成功完成代码签入后，通过自动触发构建持续集成；
- 测试自动化脚本可以按照作业计划执行，实现持续交付；
- 通过报告和仪表板持续监测程序发布质量；
- 可通过自动化的缺陷识别和解决方案，帮助用户快速响应变更；
- 可以提供基于关键绩效指标（KPI）的有价值的报告，以使用户快速做出决策；
- 通过跟踪发布流水线实现持续交付。

0 云原生应用的技术手段-持续集成、持续交付

持续集成是一种软件开发的实践方法，它要求团队成员经常整合他们的工作成果（通常是程序代码）。通常情况下，团队成员中的每人每天至少提交一次自己的代码到代码仓库做集成构建，这样对于整个项目而言，每天就会有多次集成构建。

持续交付是一种以可持续的方式安全快速地将所有类型的软件变更（包括新功能开发、配置更改、Bug修复等）转化为生产环节下的工作产品交付给用户直接使用的软件过程控制方法，它的最终目标是将变更直接部署到生产环境。

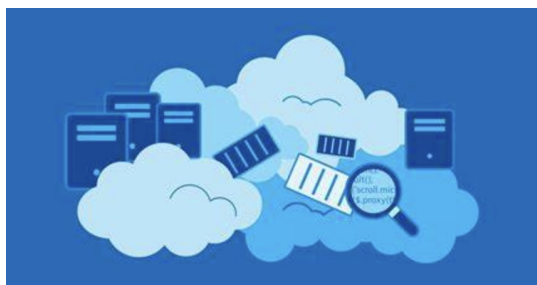
0 云原生应用的技术手段-容器

容器技术与虚拟机技术相比，拥有更高的资源使用效率，因为它并不需要为每个应用分配单独的操作系统，所以实例规模更小、创建和迁移速度也更快。相对于虚拟机，单个操作系统能够承载更多的容器。

容器化最大的好处是保持运行环境的一致性，只要应用可以打包成容器镜像（通常使用Docker容器），就可以一次编译后，在各处运行。

同时，容器也可以作为应用运行的最小组件来部署，且更适合作为无状态应用运行。结合容器编排工具（如Kubernetes）将大大增强系统的扩展性和自愈能力，轻松应对大流量下的高并发场景，加快业务的迭代速度。

Kubernetes作为CNCF（云原生计算基金会）成员的核心，本身就是与云原生应用的理念紧密结合的产物。



0 云原生归纳

充分利用云计算技术的优势：采用云端优先策略，从云服务中获取最大价值；

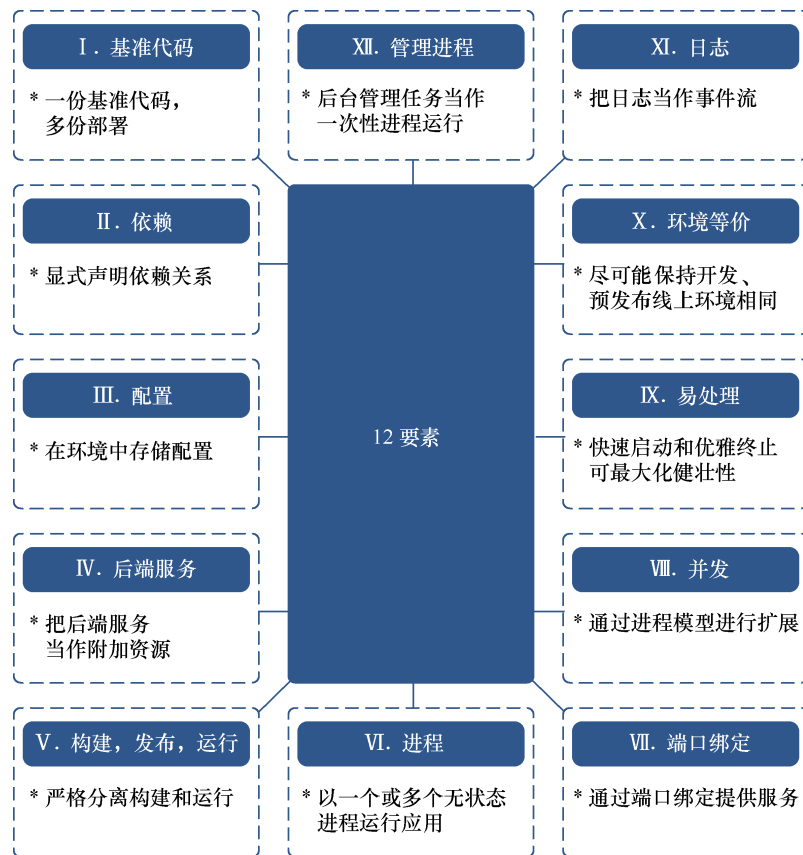
- 实现快速、敏捷、频繁的交付模式；
- 通过技术创新更多地扩展云计算技术的边界。

云原生中包含的不同思想，与其所解释的云上应用架构应该具备的特性几乎是一一对应的：

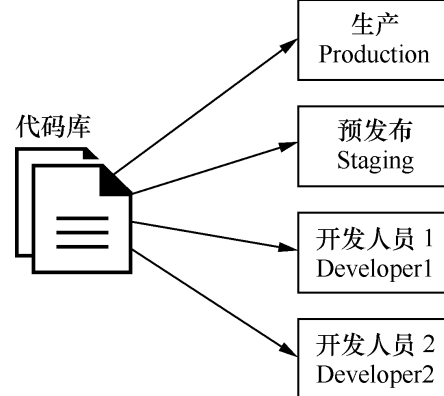
- DevOps、持续交付对应更快的上线速度，即敏捷性；
- 微服务对应可扩展性及故障可恢复性；
- 敏捷基础设施实现了扩展能力的资源层支持；
- 康威定律在组织结构和流程上确保架构特性能够快速实施。

0 云原生应用开发实践的12要素

1. 一份代码库与多份部署
2. 显式声明依赖关系
3. 在环境中存储配置
4. 把后端服务当作附加资源
5. 严格分离构建和运行
6. 以一个或多个无状态进程运行应用
7. 通过端口绑定提供服务
8. 通过进程模型进行扩展
9. 快速启动和优雅终止可最大化健壮性
10. 尽可能保持开发与预发布线上环境相同
11. 把日志当作事件流
12. 后台管理任务当作一次性进程运行



0 云原生应用开发实践的12要素



1. 一份代码库与多份部署

- 12要素应用通常会使用版本控制系统加以管理，如Git、Mercurial、Subversion。一个用来跟踪代码同时存在多份部署。所有部署的代码库相同，但每份部署可以使用其不同所有修订版本的数据库被称作代码库（code repository、code repo、repo）。
- 代码库和应用之间总是保持一一对应的关系。
- 一旦有多个代码库，就不能称为一个应用，而是一个分布式系统。分布式系统中的每一个组件都是一个应用，每一个应用可以分别使用12要素进行开发。
- 多个应用共享一个代码库是有悖12要素原则的。解决方案是将共享的代码拆分为独立的类库，然后使用依赖管理策略去加载它们。
- 尽管每个应用只对应一个代码库，但可以有多个版本。

0 云原生应用开发实践的12要素

2. 显式声明依赖关系

- 大多数编程语言都会提供一个打包系统，为各个类库提供打包服务，就像 Perl 的CPAN或是 Ruby的Rubygems。通过打包系统安装类库可以是系统级的（称之为“site packages”），或仅供某个应用程序使用，部署在相应的目录中（称之为“vendoring”或“bundling”）。
- 12要素原则下的应用程序不会隐式依赖系统级的类库。它一定通过“依赖清单”，确切地声明所有依赖项。
- 显式声明依赖的优点之一是为新进开发者简化了环境配置流程。新进开发者可以找出应用程序的代码库，安装编程语言环境和它对应的依赖管理工具，只需通过一个“构建命令”就能安装所有的依赖项开始工作。

0 云原生应用开发实践的12要素

3. 在环境中存储配置

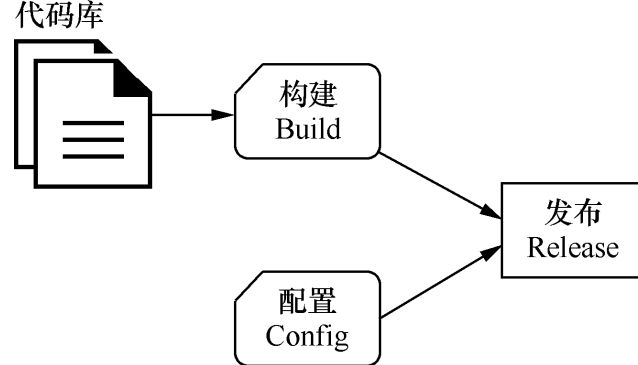
- 通常，应用的配置在不同部署（预发布、生产环境、开发环境等）间会有很大差异。有些应用在代码中使用常量保存配置，这与12要素所要求的代码和配置严格分离显然不符。配置文件在各部署间存在很大差异，代码却完全一致。
- 判断一个应用是否正确地将配置排除在代码之外，一个简单的方法是看该应用的代码库是否可以立刻开源，而不用担心会暴露任何敏感的信息。另外一个解决方法是使用配置文件，但不把它们纳入版本控制系统，就像Rails的config/ database.yml。
- 12要素推荐将应用的配置存储于环境变量中（env vars, env）。环境变量可以非常方便地在不同的部署间做修改，却不用改一行代码；与配置文件不同，不小心把它们签入代码库的概率微乎其微；与一些传统的解决配置问题的机制（例如Java的属性配置文件）相比，环境变量与语言和系统无关。

0 云原生应用开发实践的12要素

4. 把后端服务当作附加资源

- 后端服务是指程序运行所需要的通过网络调用的各种服务，如数据库（MySQL, CouchDB）、消息/队列系统（RabbitMQ, Beanstalkd）、SMTP邮件发送服务（Postfix），以及缓存系统（Memcached）。
- 类似数据库的后端服务，通常由部署应用程序的系统管理员一起管理。
- 12要素应用不会区别对待本地或第三方服务。对应用程序而言，两种都是附加资源，都可以通过一个URL或是其他存储在配置中的服务定位/服务证书来获取数据。
- 每个不同的后端服务是一个资源。部署可以按需加载或卸载资源。

0 云原生应用开发实践的12要素



5. 严格分离构建和运行

- 代码库转化为一份部署（非开发环境）需要以下三个阶段
- （1）构建阶段是指将代码仓库转化为可执行包的过程。构建时会使用指定版本的代码，获取和打包依赖项，编译成二进制文件和资源文件。
- （2）发布阶段会将构建的结果和当前部署所需配置相结合，并能够立刻在运行环境中投入使用。
- （3）运行阶段（或者说“运行时”）是指针对选定的发布版本，在执行环境中启动一系列应用程序进程。
- 12要素应用严格区分构建、发布、运行三个步骤。每一个发布版本必须对应一个唯一的发布ID，例如可以使用发布时的时间戳（2011-04-06-20:32:17）或是一个增长的数字（v100）。新的代码在部署之前，需要开发人员触发构建操作。但是，运行阶段不一定需要人为触发，而是可以自动进行

0 云原生应用开发实践的12要素

6. 以一个或多个无状态进程运行应用

- 运行环境中，应用程序通常是以一个或多个进程运行的。
- 12要素应用的进程必须无状态且无共享。任何需要持久化的数据都要存储在后端服务内，例如数据库。
- 源文件打包工具（Jammit、django-compressor）使用文件系统来缓存编译过的源文件。12要素应用更倾向于在构建步骤执行此操作（如Rails资源管道），而不是在运行阶段。
- 一些互联网系统依赖于“黏性Session”，是指将用户Session中的数据缓存至某进程的内存中，并将同一用户的后续请求路由到同一个进程。黏性Session是12要素极力反对的。Session中的数据应该保存在诸如Memcached或Redis这样的带有过期时间的缓存中。

0 云原生应用开发实践的12要素

7. 通过端口绑定提供服务

- 互联网应用有时会运行于服务器的容器之中。
- 12要素应用完全自我加载而不依赖于任何网络服务器就可以创建一个面向网络的服务。互联网应用通过端口绑定来提供服务，并监听发送至该端口的请求。
- HTTP并不是唯一可以由端口绑定提供的服务。几乎所有服务器软件都可以通过进程绑定端口来等待请求。例如，使用XMPP的ejabberd，以及使用Redis 协议的Redis。
- 还要指出的是，端口绑定这种方式意味着一个应用可以成为另外一个应用的后端服务，调用方将服务方提供的相应URL当作资源存入配置以备将来调用。

0 云原生应用开发实践的12要素

8. 通过进程模型进行扩展

- 任何计算机程序一旦启动，就会生成一个或多个进程。互联网应用采用多进程运行方式。
- 在12要素应用中，进程是一等公民。12要素应用的进程主要借鉴于UNIX守护进程模型。开发人员可以运用这个模型去设计应用架构，将不同的工作分配给不同类型的进程。
- 这其中并不包括个别较为特殊的进程，例如通过虚拟机的线程处理并发的内部运算，或是使用诸如EventMachine、Twisted、Node.js的异步事件触发模型。
- 12要素应用的进程所具备的无共享、水平分区的特性意味着添加并发应用会变得简单而稳妥。12要素应用的进程不需要守护进程或是写入PID文件。相反的，应该借助操作系统的进程管理器来管理输出流

0 云原生应用开发实践的12要素

9. 快速启动和优雅终止可最大化健壮性

- 12要素应用的进程是易处理（Disposable）的，意思是它们可以瞬间开启或停止。这有利于快速、弹性地伸缩应用，迅速部署变化的代码或配置，稳健地部署应用。
- 进程应当追求最少启动时间。理想状态下，进程从输入命令到真正启动并等待请求的时间应该很短。
- 进程一旦接收到终止信号（Sigterm）就会优雅终止。就网络进程而言，优雅终止是指停止监听服务的端口，即拒绝所有新的请求，并继续执行当前已接收的请求，然后退出。
- 进程还应当在面对突然死亡时保持健壮，例如底层硬件故障。
- 无论如何，12要素应用都应该可以设计能够应对意外的、不优雅的终结。Crash-only design将这种概念转化为合乎逻辑的理论。

0 云原生应用开发实践的12要素

10. 尽可能保持环境相同

- 从以往经验来看，开发环境（即开发人员的本地部署）和线上环境（外部用户访问的真实部署）之间存在着很多差异。这些差异表现在以下三个方面。
- 时间差异：开发人员正在编写的代码可能需要几天、几周，甚至几个月才会上线。
- 人员差异：开发人员编写代码，运维人员部署代码。
- 工具差异：开发人员使用Nginx、SQLite、OS X，而线上环境使用Apache、MySQL及Linux。
- 12要素应用要想做到持续部署就必须缩小本地与线上差异。缩小时间差异：开发人员可以几小时，甚至几分钟就部署完代码。
- 缩小人员差异：开发人员不只是编写代码，更应该密切参与部署过程以及关注代码在线上的表现。
- 缩小工具差异：尽量保证开发环境以及线上环境的一致性。

0 云原生应用开发实践的12要素

11. 把日志当作事件流

- 日志使应用程序的运行变得透明。日志应该是事件流的汇总，即将所有运行中进程和后端服务的输出流按照时间顺序收集起来。
- 12要素应用本身并不考虑存储自己的输出流，因此不应该试图去写或者管理日志文件。相反，每一个运行的进程都会对应直接的标准输出（Stdout）事件流。这些事件流可以输出至文件，或者在终端实时观察。最重要的，输出流可以发送到Splunk这样的日志索引及分析系统，或Hadoop/Hive这样的通用数据存储系统。这些系统为查看应用的历史活动提供了强大而灵活的功能。

0 云原生应用开发实践的12要素

12. 后台管理任务当作一次性进程运行

- 进程构成 (Process Formation) 是指用来处理应用的常规业务（例如处理Web请求）的一组进程。与常规业务不同，开发人员经常希望执行一些管理或维护应用的一次性任务。
- 一次性管理进程应该和正常的常驻进程一样使用同样的环境，和任何其他的进程一样使用相同的代码和配置，基于某个发布版本运行。后台管理代码应该随其他应用程序代码一起发布，从而避免同步问题。
- 所有进程类型应该使用同样的依赖隔离技术。
- 12要素应用尤其青睐那些提供了REPL shell的语言，因为这会让运行一次性脚本变简单。在本地部署中，开发人员直接在命令行使用shell命令调用一次性管理进程。

0 云原生应用开发的原则

云原生的开发范式是软件开发演进的一种新型范式，它不仅仅是将应用程序迁移和移植到云平台上运行，更加关注如何利用云计算并最大限度地发挥其优势。为了实现这一目标，在生产和开发过程中，软件开发相关的部门都需要认真关注如何使用云服务，进而关注并实践如何构建云原生应用。

0 云原生应用开发的原则-云服务优先策略

原则：云服务优先策略（Cloud-First）。

描述：在评估技术解决方案中的服务或组件时，首先要考察目前市面上是否有可用的云服务功能，并优先考虑使用最适合用户需求的云服务。

理由：将需要自己负责全新开发的软件模块数量降到最低、最合理水平。

参考建议：

- 云原生的服务应该部署在云端，除非受限于一些特殊的环境因素，如安全、合规问题，或者受限于特殊的网络、集成需求问题。
- SaaS适用于一些大中型应用功能，同时也支持自定义和个性化设置，这一点相对于版权许可软件来说更具灵活性。
- 必须权衡考虑版权许可软件和开源软件。

0 云原生应用开发的原则-基础设施即代码

原则：基础设施即代码（Infrastructure as Code, IaC）。

描述：以处理应用程序代码相同的方式来管理基础设施配置以及工作流的定义。

理由：通过API的方式来构建环境，提供管理和执行运行环境工作流的工具，这使得环境配置可以视为软件功能的一部分。

参考建议：

- 需要使用支持IaC的工具；
- 需要为应用软件及其运行环境编写相应的测试脚本；
- 环境的准备和配置不可以通过手动操作的方式进行。

0 云原生应用开发的原则-敏捷交付

原则：敏捷交付（Agile Delivery）。

描述：在交付过程的各个阶段争取敏捷，包括开发前的项目启动和计划阶段，以及开发后发布管理和运维管理阶段。

基本原理：敏捷软件开发过程通常能使产品更快地投入生产，但如果开发过程控制过于死板，项目开发就无法敏捷，只有力争各个阶段保持敏捷，才可以最大限度地提高效益。

理由：

前期开发规划应充分考虑项目迭代周期与开发交付周期之间的呼应关系，使软件的开发过程适应敏捷开发的过程控制方式。

必须设定一个初始交付目标，这个交付物必须是可以运行的工作成果。

随着业务目标的调整，对于开发过程中的需求变更应该抱有开放的态度，拥抱变化。

开发团队和运维团队紧密合作，力求做到频繁发布，充分采用DevOps的开发理念。

快速试错，避免冗长的QA测试环节，最大程度地降低交付风险。

0 云原生应用开发的原则-自动化交付

原则：自动化交付原则（Delivery Automation）。

描述：力求在开发运维过程中做到从构建到发布的全自动化。

理由：实现软件构建、环境准备、测试和部署的自动化能力可以使得产品在加速市场化的过程中占据绝对的优势。

参考建议：

- 这个原则建立在前面的基础设施即代码的原则之上。
- 自动化测试工具是必需的。
- “快速试错”的方法是为了加快部署和自动化生产。
- 应该设计一个监控系统 and 回滚计划，以便快速检测和回退有问题的版本，而不用等待错误修复。

0 云原生应用开发的原则-基于服务架构

原则：基于服务架构（Service-Based Architecture）。

描述：必须按照既定的项目目标和期望的特点来遵循各种形式的基于服务的体系结构（SBA）。

理由：所有形式的基于服务的体系结构都有其优点，应该加以利用。

虽然在选择一种具体的形式时需要权衡，但应该考虑和评估各种服务形式，为给定的解决方案确定最合适的架构方法。

参考建议：

- 为了确定基于服务的体系结构最适合的应用，在软件开发生命周期的早期就需要进行一些分析。
- 所有形式的SBA都要求按API的规范化开发。
- 应采用API优先开发战略。
- 需要考虑API的接口风格的标准化。
- 需要考虑API的接口的安全性，并采取相应的措施保障API不暴露给不安全或不受信的网络。

0 云原生应用开发的原则-12要素应用

原则：12要素应用（Twelve-Factor Applications）。

描述：遵循最佳实践（如12要素应用原则），开发云原生应用程序。

理由：一些组织多年来一直致力于开发云原生应用程序，并开始记录最佳实践，需要吸取别人的教训，并在适当的时候采取最佳作法。

参考建议：

- 构建过程，发布过程和配置管理实践可能受某些最佳实践的影响。
- 一些最佳实践会影响应用程序的部署和管理方式，因此可能有必要查看运营团队成员的最佳实践。

0 云原生实践：Node.js云原生应用开发

随着云计算的火热发展，云原生应用的开发实践也成为一种趋势，目前已有
很多厂商都提供了支持云原生应用开发的基础设施。

使用DaoCloud的服务，以一个Node.js的Hello World的简单程序来讲述实
践云原生的应用开发过程。

1. 第一步：准备Node.js程序

与之前的Node.js开发流程一样，首先要准备好Node.js应用的配置。

```
$ mkdir -p node-docker-example
```

```
$ cd node-docker-example
```

开始编写主要的程序。

```
// app.js
```

2. 第二步：添加配置文件

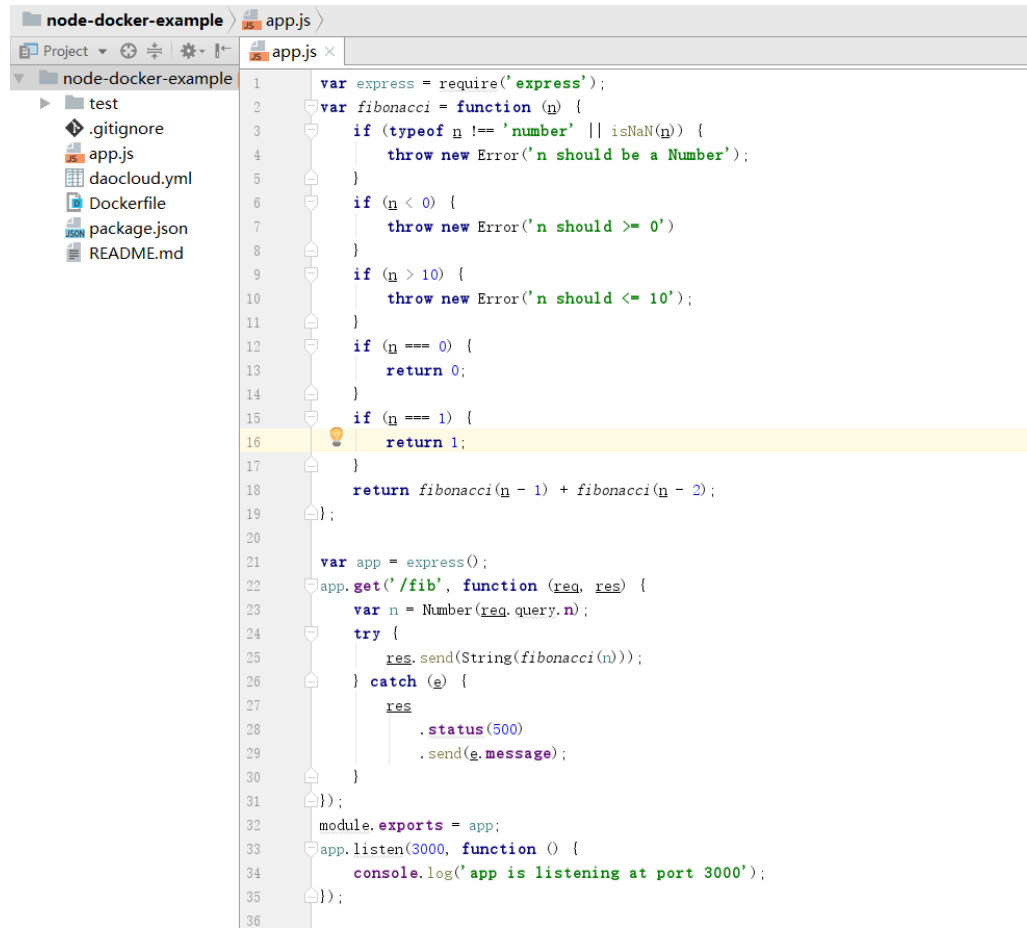
我们可以给项目添加Node.js Package配置文件。

```
$ npm init -y
```

3. 第三步：编写Dockerfile

Dockerfile是一个Docker镜像的核心部件，所有的构建、运行入口、
容器配置都依赖它。从DockerHub拉取一个Node.js的官方Docker镜像，
作为基础镜像。

演示程序代码结构

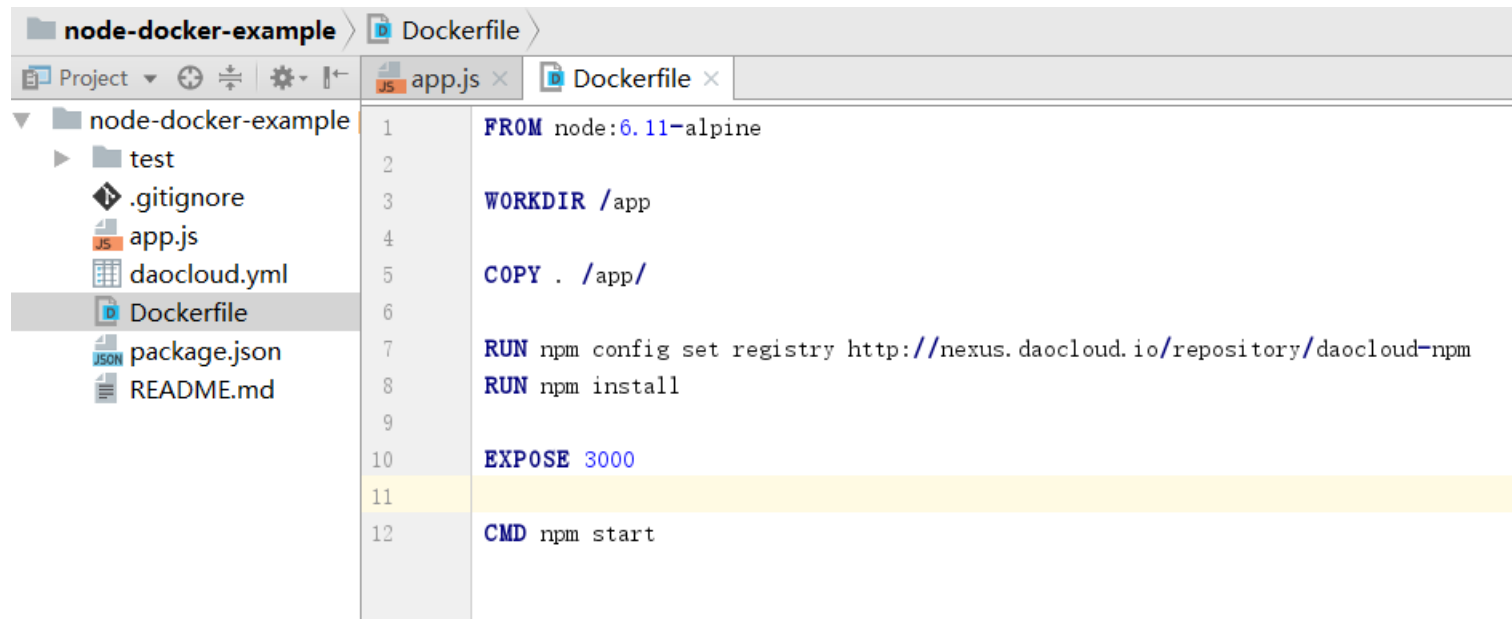


The screenshot displays a code editor with a project named 'node-docker-example'. The left sidebar shows the file structure, including a 'test' directory and files like '.gitignore', 'app.js', 'daocloud.yml', 'Dockerfile', 'package.json', and 'README.md'. The main editor area shows the content of 'app.js', which is a Node.js application using Express.js to implement a Fibonacci sequence API.

```
1  var express = require('express');
2  var fibonacci = function (n) {
3    if (typeof n !== 'number' || isNaN(n)) {
4      throw new Error('n should be a Number');
5    }
6    if (n < 0) {
7      throw new Error('n should >= 0');
8    }
9    if (n > 10) {
10     throw new Error('n should <= 10');
11   }
12   if (n === 0) {
13     return 0;
14   }
15   if (n === 1) {
16     return 1;
17   }
18   return fibonacci(n - 1) + fibonacci(n - 2);
19 };
20
21 var app = express();
22 app.get('/fib', function (req, res) {
23   var n = Number(req.query.n);
24   try {
25     res.send(String(fibonacci(n)));
26   } catch (e) {
27     res
28       .status(500)
29       .send(e.message);
30   }
31 });
32 module.exports = app;
33 app.listen(3000, function () {
34   console.log('app is listening at port 3000');
35 });
36
```



Dockerfile内容



The screenshot shows an IDE window titled 'node-docker-example' with a 'Dockerfile' tab active. The left sidebar displays the project structure, including a 'test' folder, '.gitignore', 'app.js', 'daocloud.yml', 'Dockerfile', 'package.json', and 'README.md'. The main editor area shows the content of the Dockerfile, which is a multi-line script for building and running a container. The lines are numbered 1 through 12. Line 11 is highlighted in yellow.

```
1 FROM node:6.11-alpine
2
3 WORKDIR /app
4
5 COPY . /app/
6
7 RUN npm config set registry http://nexus.daocloud.io/repository/daocloud-npm
8 RUN npm install
9
10 EXPOSE 3000
11
12 CMD npm start
```



新建DaoCloud Service项目

← → ↻ 安全 | https://dashboard.daocloud.io/build-flows

DaoCloud Services

DevOps

项目

交付中心

镜像仓库

收藏夹

发现镜像

应用平台

自有主机

设置

用户中心

创建新项目 共 2 个项目

搜索

项目名称	最近更新	代码仓库	执行状态
 python-mysql-sample master-07f3d43	9月前	 Super-Warrior/python-mysql-sample	执行成功 查看详情
 example master-a2c564e	11月前	 chunchill/dao-2048	执行成功 查看详情

< 上一页 下一页 > 第 1 页, 共 1 页



关联GitHub项目

DaoCloud Services

DevOps

项目

交付中心

镜像仓库

收藏夹

发现镜像

应用平台

自有主机

设置

用户中心

项目名称

nodejs-demo

项目镜像：daocloud.io/jasper_chiu/nodejs-demo

设置代码源

代码源指定了您项目的代码仓库位置，关联代码源后，您对代码源的操作会自动触发项目的相应动作比如持续集成和镜像构建。

Github

Bitbucket

Coding

GitLab

Git 地址



JohnGriffithLondon

ry-bpms ▾



NatureFeng

nodeinpractice ▾



PYDebug

SVG ▾



Super-Warrior

BA ▾



chunchill

nodejs-sample ▾



jiashuhui

jiashuhui.github.com ▾



定义CI流程

DaoCloud Services

DevOps

交付中心

应用平台

设置

项目

镜像仓库

收藏夹

发现镜像

自有主机

用户中心

nodejs-demo

镜像: nodejs-demo 代码源: chunchill/nodejs-sample

push 代码触发或 手动触发

执行记录 流程定义 设置

测试阶段

构建阶段

默认测试任务

test

添加并行任务

默认构建任务

build

添加并行任务

全局镜像

环境变量

使用 submodule

构建集群 IP

通过 yaml 快捷编辑

切换至本地 yaml



daocloud.yml内容

DaoCloud Services

DevOps

项目

交付中心

镜像仓库

收藏夹

发现镜像

应用平台

自有主机

设置

用户中心

nodesjs-demo

镜像: nodesjs-demo 代码源: chunchill/nodejs-sample

push 代码触发或 手动触发

执行记录 流程定义 设置

```
version: 3
stages:
  - 测试阶段
  - 构建阶段
默认构建任务:
  label: build
  stage: 构建阶段
  job_type: image_build
  only:
    branches:
      - .*
    tags:
      - .*
  build_dir: /
  cache: true
  dockerfile_path: /Dockerfile
默认测试任务:
  label: test
  stage: 测试阶段
  job_type: test
  only:
    branches:
      - .*
    tags:
      - .*
  image: node:6.11.0-wheezy
  script:
    - npm install
```

使用 submodule

构建集群 IP

切换至云端定义



手动触发流水线



流水线执行过程中

DaoCloud Services

DevOps

项目

交付中心

镜像仓库

收藏夹

发现镜像

应用平台

自有主机

设置

用户中心

创建新项目 共 3 个项目

搜索

项目名称	最近更新	代码仓库	执行状态	
nodejs-demo master-ba560f6	大约1分钟前	chunchill/nodejs-sample	正在执行	查看详情
python-mysql-sample master-07f3d43	9月前	Super-Warrior/python-mysql-sample	执行成功	查看详情
example master-a2c564e	11月前	chunchill/dao-2048	执行成功	查看详情

< 上一页

下一页 >

第 1 页, 共 1 页



流水线执行成功

DaoCloud Services

DevOps

项目

交付中心

镜像仓库

收藏夹

发现镜像

应用平台

自有主机

设置

用户中心

创建新项目

共 3 个项目

搜索

项目名称	最近更新	代码仓库	执行状态	
nodejs-demo master-ba560f6	2分钟前	chunchill/nodejs-sample	执行成功	查看详情
python-mysql-sample master-07f3d43	9月前	Super-Warrior/python-mysql-sample	执行成功	查看详情
example master-a2c564e	11月前	chunchill/dao-2048	执行成功	查看详情

上一页

下一页

第 1 页, 共 1 页



查看构建状态

DaoCloud Services

DevOps

项目

交付中心

镜像仓库

收藏夹

发现镜像

应用平台

自有主机

设置

用户中心

 **nodejs-demo**
镜像: [nodejs-demo](#) 代码源: [chunchill/nodejs-sample](#)

push 代码触发或 手动触发

执行记录

流程定义

设置

执行记录编号	commit	commit message	状态	时间
master-ba560f6 👤 jasper_chiu手动触发	📁 master 👉 ba560f6	-	✅ 执行成功	耗时1分34秒 2分钟前提交



部署构建好的版本

The screenshot displays the DaoCloud Services interface. On the left is a dark sidebar with navigation options: DevOps, 项目 (Projects), 交付中心 (Delivery Center), 镜像仓库 (Image Registry), 收藏夹 (Favorites), 发现镜像 (Discover Images), 应用平台 (Application Platform), 自有主机 (Self-owned Hosts), 集群管理 (Cluster Management), 应用 (Applications), Stack, 设置 (Settings), and 用户中心 (User Center). The '镜像仓库' option is highlighted with a red box.

The main content area shows a list of images under the heading '本地 Push 镜像' (Local Push Image) with a sub-header '共 3 个镜像' (Total 3 images). A search bar labeled '搜索镜像' (Search Image) is in the top right. The list contains three items:

Image Name	Repository	Version	Action
nodejs-demo 更新于 2分钟前	镜像: daocloud.io/jasper_chiu/nodejs-demo 版本: master-ba560f6	部署最新版本	
python-mysql-sample 更新于 9月前	镜像: daocloud.io/jasper_chiu/python-mysql-sample 版本: master-07f3d43	部署最新版本	
example 更新于 11月前	镜像: daocloud.io/jasper_chiu/example 版本: master-a2c564e	部署最新版本	

The 'nodejs-demo' row is highlighted with a red box, and its '部署最新版本' button is circled in red.



对接部署服务器

DaoCloud Services

DevOps

项目

交付中心

镜像仓库

收藏夹

发现镜像

应用平台

自有主机

集群管理

应用

Stack

设置

用户中心

基本信息

应用设置

应用信息和运行环境

应用名称

nodejs-app

镜像

daocloud.io/jasper_chiu/nodejs-demo

版本号

master-ba560f6

运行环境

主机或集群

+ 接入主机

快速开始，先试试胶囊主机吧！

- 胶囊主机自带 **Docker** 运行环境，并自动接入 DaoCloud 容器管理平台。
- 每个胶囊主机可以免费体验 **120 分钟**。到期后自动消失，也可以再次体验。
- 发布到胶囊主机中的应用信息会保留并可以自动迁移。

一键接入

下一步

对接DaoCloud胶囊主机

运行环境

主机或集群

+ 接入主机

快速开始，先试试胶囊主机吧！

- 胶囊主机自带 **Docker** 运行环境，并自动接入 DaoCloud 容器管理平台。
- 每个胶囊主机可以免费体验 **120 分钟**。到期后自动消失，也可以再次体验。
- 发布到胶囊主机中的应用信息会保留并可以自动迁移。

⚙ 正在接入

预计还剩 44 s



随机生成DaoCloud胶囊主机

➤ 基本信息

➤ 应用设置

应用信息和运行环境

应用名称

nodejs-app

镜像

daocloud.io/jasper_chiu/nodejs-demo

版本号

master-ba560f6

运行环境

主机或集群

主机

Try_DaoCloud_1
IP: 10.23.223.147,172.17.0.1

下一步



配置映射端口

DaoCloud Services

DevOps

项目

交付中心

镜像仓库

收藏夹

发现镜像

应用平台

自有主机

集群管理

应用

Stack

设置

用户中心

基本信息

应用设置

应用设置

端口	容器端口	协议	主机端口	是否发布	
	3000	TCP	动态端口	是	

添加容器端口

容器端口会映射到主机端口上。

Volumes

容器路径	主机路径	是否可写
您还没有添加 Volume		

添加 Volume

自定义环境变量

手动输入

导入 YAML

键	-	值
---	---	---

显示高级设置

立即部署

返回上一步



运行成功结果

DaoCloud Services

DevOps

项目

交付中心

镜像仓库

收藏夹

发现镜像

应用平台

自有主机

集群管理

应用

Stack

设置

用户中心

nodejs-app

运行中

创建于 1分钟之内

所属镜像: daocloud.io/jasper_chiu/nodejs-demo

镜像版本: master-ba560f6

运行环境: 我的主机

主机: Try_DaoCloud_1

容器

配置

YAML

监控

日志

事件

云隧道

发布

迁移

设置

停止

重新部署

2018-01-21 21:35:50: eb03949b4ac3: Extracting [=====>] 6.226 MB/15.44 MB

2018-01-21 21:35:51: eb03949b4ac3: Extracting [=====>] 14.58 MB/15.44 MB

2018-01-21 21:35:51: eb03949b4ac3: Extracting [=====>] 15.07 MB/15.44 MB

2018-01-21 21:35:51: eb03949b4ac3: Extracting [=====>] 15.44 MB/15.44 MB

2018-01-21 21:35:51: eb03949b4ac3: Pull complete

2018-01-21 21:35:51: 7cb05f3b6429: Extracting [=>] 32.77 kB/1.016 MB

2018-01-21 21:35:52: 663b3d1fcb84: Extracting [=====>] 92 B/92 B

2018-01-21 21:35:53: dd197113ec57: Extracting [=====>] 1.697 kB/1.697 kB

2018-01-21 21:35:53: dd197113ec57: Pull complete

2018-01-21 21:35:53: 014895872457: Extracting [=====>] 852 kB/2.813 MB

2018-01-21 21:35:53: 014895872457: Extracting [=====>] 2.556 MB/2.813 MB

2018-01-21 21:35:53: Status: Downloaded newer image for daocloud.io/jasper_chiu/nodejs-demo:master-ba560f6

2018-01-21 21:35:55: Creating dao_nodejs-app-1...

2018-01-21 21:35:57:

2018-01-21 21:35:57: > nodejs-sample@0.0.1 start /app

2018-01-21 21:35:57: > node app.js

2018-01-21 21:35:57:

2018-01-21 21:35:57: app is listening at port 3000



运行成功后状态

DaoCloud Services

DevOps

项目

交付中心

镜像仓库

收藏夹

发现镜像

应用平台

自有主机

集群管理

应用

Stack

设置

用户中心

Try_DaoCloud_1

状态: 正常

IP: 10.23.223.147, 172.17.0.1 外网: 52.80.161.165

主机名: ip-10-23-223-147

Docker 1.13.0: 运行正常

连接主机: [ssh ubuntu@52.80.161.165](#) 密码: 61c4a9055d58a31e

操作系统: Ubuntu 14.04.2 LTS

集群: 默认

还能试用: 01:56:07

容器 镜像 网络 Volume 监控 设置

共 1 个容器, 已选中 0 个容器

容器	镜像	端口	状态
dao_nodejs-app_1 9f77198efaa1	daocloud.io/jasper_chiu/nodejs-demo:master-ba560f6 /bin/sh -c 'npm start'	32768 -> 3000	Up Less than a second 查看详情

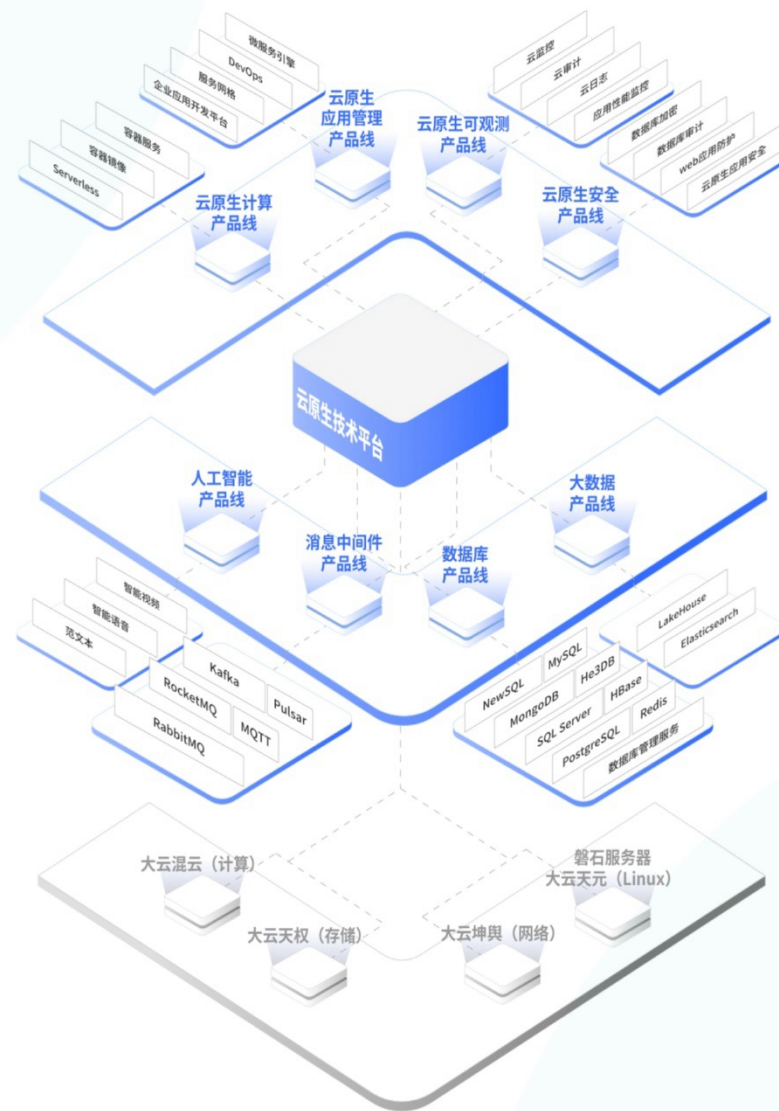


1 移动云云原生

移动云秉承“算力网络”理念，自主研发星罗分布式云原生技术平台，以云原生开源技术及标准为基础，充分发挥自身产品与技术创新优势，联合产业生态，为客户提供全栈云原生产品家族，帮助客户实现应用敏捷、业务智能，安全可信，面向未来持续演进。



2 移动云云原生全景图

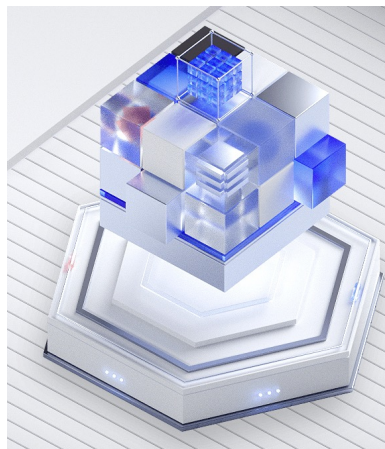


3 云原生产品家族

- **容器**：容器服务、容器镜像服务、函数计算
- **开发者服务**：研发效能工具、微服务引擎、企业应用开发平台
- **数据库**：云数据库Mysql、云数据库Redis、云数据库PostgreSQL、文档数据库、云数据库SQL Server、云数据库NewSQL、云原生数据库He3DB、数据库管理服务、云数据库HBase
- **消息队列**：消息队列RocketMQ、消息队列Kafka、消息队列Pulsar、消息队列MQTT、消息队列RabbitMQ
- **大数据**：Elasticsearch、云原生大数据分析LakeHouse
- **人工智能**：人脸与人体识别、自然语言处理、通用文字识别

3 云原生产品家族-容器

容器服务



容器服务 KCS (Kubernetes Container Service) 提供了高性能高可靠的容器应用管理能力，支持容器化应用全生命周期管理，支持一键部署集群，并提供丰富的集群管理能力，简化云端容器化应用运行环境的搭建，提供完整的云端应用运行环境。

产品优势：

简单易用：一键创建容器集群，一站式部署运维容器化应用，实现应用全生命周期管理。

安全可靠：集群控制面支持3 Master HA高可用，当其中某个Master节点故障时，集群依然可用，从而保障您的业务高可用。

开放兼容：Kubernetes服务提供商资质（KCSP），产品完全兼容Kubernetes/Docker社区原生版本。

集群规模：单个集群可管理节点数量达千级。

3 云原生产品家族-容器

产品功能：

一站式部署和运维：一键创建Kubernetes集群，自动化部署、运维容器应用，整个生命周期都在容器服务内一站式完成。

丰富的应用调度策略：支持工作负载与节点以及负载间的多种亲和反亲和调度策略，在高性能和高可靠中找到平衡点。

兼容原生Kubernetes和Docker：与社区最新版本保持紧密同步，完全兼容原生API和命令。

高可用业务不中断：集群控制面HA，保障业务高可用。

灵活的弹性伸缩政策：支持集群节点和工作负载2级的弹性伸缩，用户可自由组合策略以应对多变的突发浪涌。

应用场景

微服务架构转型：助力传统应用向微服务架构转变，提高系统的灵活性和可维护性。

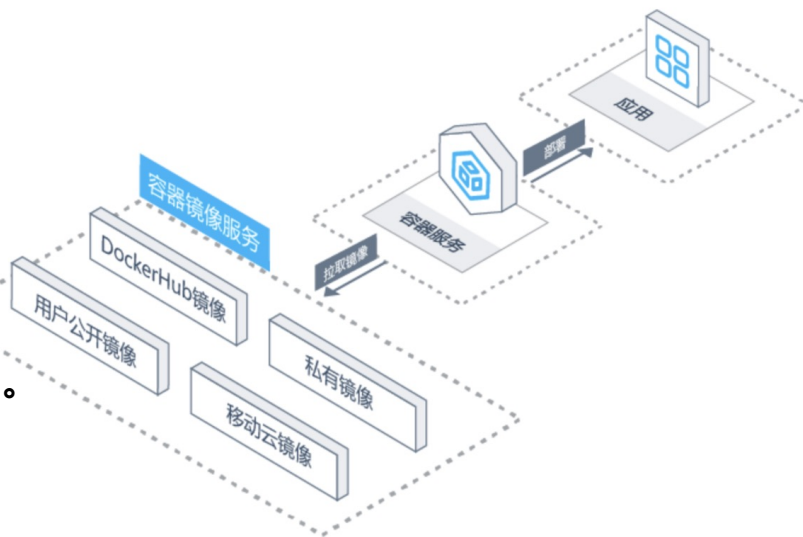
加速敏捷开发与产品迭代：通过容器化部署，加快开发测试流程，缩短产品上市时间。

传统单体系统现代化：将传统的单一庞大系统合理拆分为松耦合、独立运行的微服务，每个微服务应用均可存储在移动云容器镜像仓库中，实现统一管理和高效分发。容器服务不仅支持微服务应用的灵活调度、自动编排、一键部署，还具备灰度发布能力，进一步提升服务的稳定性和更新效率。

3 云原生产品家族-容器

容器镜像服务

容器镜像服务 CIS (Container Image Service) 是一种简单易用、安全可靠的镜像管理服务，提供容器镜像的全生命周期管理，方便用户拉取、推送、删除镜像，并且提供大量移动云镜像、DockerHub镜像和用户公开镜像，方便用户使用。



产品优势

安全可靠：提供安全加固的常用容器镜像，可对镜像进行安全扫描、镜像加签，使用移动云对象存储，保证存储可靠性。

灵活易用：通过在线制作镜像、镜像同步等功能，使得镜像构建和使用更加方便，节约用户时间。

高效分发：提供基于P2P方式支持千节点下镜像的大规模分发，分发效率提升数倍，保障容器业务的快速扩展和极速部署。

多类型资产托管：支持多架构容器镜像（Linux、ARM等架构）、Helm Chart等云原生资产的全生命周期管理。

3 云原生产品家族-容器

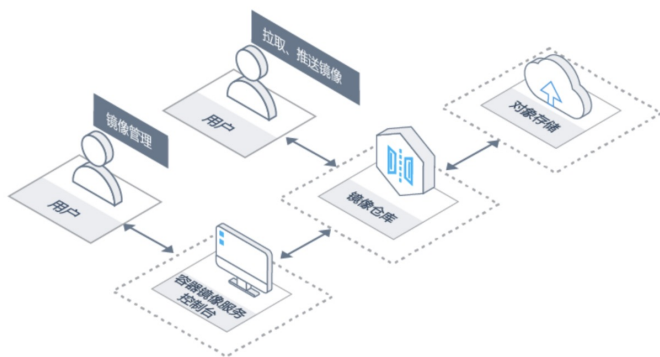
产品功能

镜像管理：提供镜像的全生命周期管理，支持推送、拉取、删除镜像，查看镜像层信息，自定义镜像描述和摘要等操作。

空间管理：提供镜像的命名空间管理，方便用户对镜像进行逻辑上的隔离和管理，用户可选择性公开镜像。

镜像收藏：用户可收藏镜像资源中的镜像，方便用户对其所关注的镜像进行管理和使用。

镜像资源：为用户提供海量镜像资源，包括移动云镜像、DockerHub镜像和用户公开镜像，方便用户收藏和拉取。



应用场景

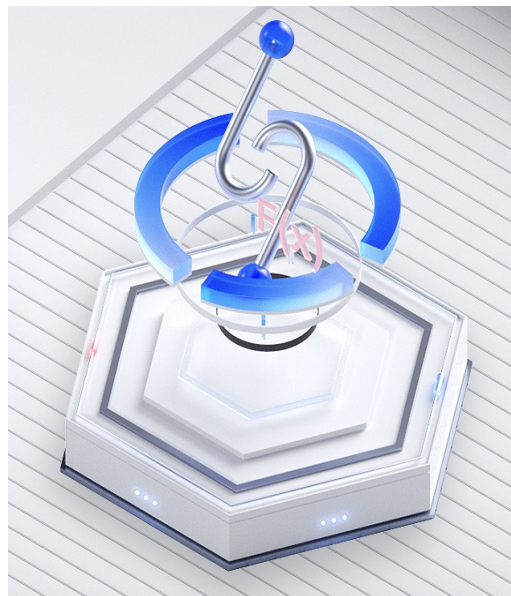
镜像生命周期管理：为用户提供私有镜像、移动云镜像、用户公开镜像及Helm Chart等多种云原生资产的全生命周期管理能力。

应用部署及更新：配合容器服务，为用户提供多种云原生资产，手动或自定义触发器等方式实现应用部署和更新。

3 云原生产品家族-容器

函数计算

移动云函数计算 SFC (Serverless Function Compute) 是一项基于Serverless技术的计算服务，提供了一种事件驱动的计算模型。



产品优势

免运维：无需购买和维护服务器等基础设施，用户能够更加专注于函数的编写，上传函数即可自动部署，提升开发和迭代速度。

高弹性：根据请求量，自动实现函数的弹性扩容，不依赖于手动配置，轻松应对业务洪峰。

低成本：根据函数的调用次数、运行时长进行计费，代码未运行时不产生费用，节约用户的使用成本。时长计量粒度为100毫秒。

多模板：提供对象存储操作、Redis缓存访问、MySQL操作以及HTTP访问等模板，用户可直接通过模板创建函数，方便快捷。

3 云原生产品家族-容器

产品功能

丰富的业务场景：触发器是触发函数执行的方式，通过HTTP、定时、对象存储、Redis等触发器可形成构建Web应用、图片处理、Redis数据处理等多种业务场景，用户可在线对函数进行配置、测试。

多语言环境：支持Java8、Python2.7/3.6、Node.js6.10/8.10、Go1.13.8多种函数运行环境，用户可通过在线编辑或上传代码包的方式管理代码。

灵活计费：根据调用次数和执行时长计费。

运维管理：为用户提供业务日志的查看、导出，并且支持函数监控，可查看函数的调用次数、调用时间等数据。

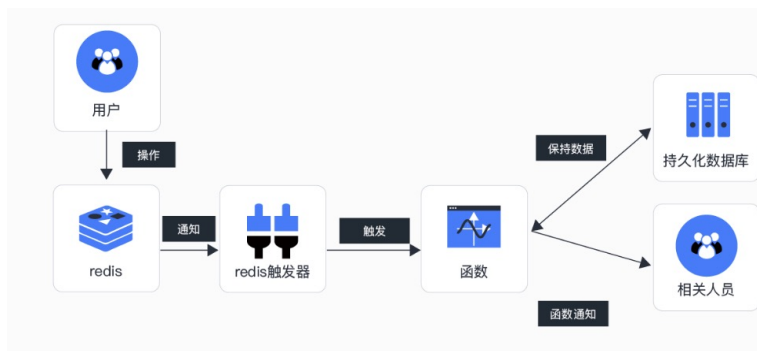
应用场景

图片处理：结合对象存储触发器，用户可编写各种图片处理函数，例如水印处理、数据抽取、图像识别等，即可对用户上传到对象存储中的图片进行自动处理。

Redis数据转储和监控：结合Redis 触发器，将Redis 数据写入持久化的数据库，或者监控特定数据的变化并通知相关人员。

搭建Web应用：结合HTTP触发器，用户只需编写业务代码即可快速搭建Web 应用。

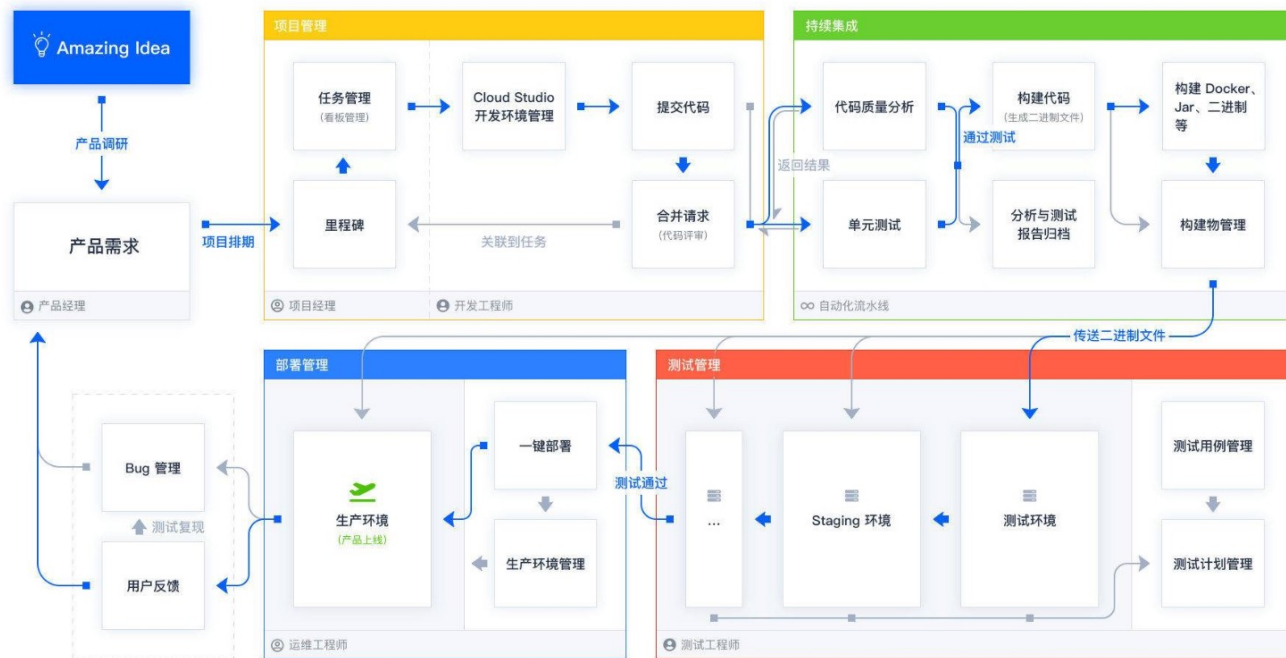
周期性数据采集：通过定时触发器，实现周期性采集应用的日志数据。



3 云原生产品家族-开发者服务

代码库管理

代码库管理是一款面向软件开发者提供的在线代码托管服务应用,提供了仓库管理、分支管理、在线管理、代码 Review、代码合并/评审、代码扫描等功能,对代码提供了安全、高效、稳定的保护。



产品优势

高可用仓库：支持 Git / SVN 两种常见版本控制工具，云端高可用，数据安全有保障。

代码评审：逐行对比、逐行评论、分支合并自动添加评审者，提供良好的代码阅读与代码审查体验。

规范版本管理：发布代码自动生成 ChangeLog，可规范版本和任务/缺陷直接关联，让源码发布与交付有标准可循。

代码浏览：支持上下、左右对比模式浏览代码，代码提交前后改动差异清晰可见。

3 云原生产品家族-开发者服务

产品功能

代码仓库管理：仓库基于分布式高性能存储。提供在线浏览、编辑等操作。支持上下、左右对比模式浏览代码，代码提交前后改动差异清晰可见。

分支管理：基于分支管理可实现版本的灵活有效控制。支持分支权限管理、设置分支管理员、设置分支的推送权限等。

代码评审：人性化评审工具，逐行对比、逐行评论，提供良好的代码阅读与代码审查体验，支持限制代码合并的权限。

协作管理：需求迭代和代码库之间高度关联，保证开发团队之间高效沟通协作。



应用场景

代码管理：代码托管支持Git、SVN两种模式，支持查看提交历史、代码对比、保护分支、配置部署公钥等功能特性，方便研发团队管理代码资源。

代码评审：开发完毕的代码可以在合并入主干分支前进行组员评审、审批等流程，并可以配置持续集成等自动化工具检测代码质量进行辅助评审。

分支管理：能清晰地查看各分支权限，针对团队成员设置分支操作权限，进行精确有效的管理，提高安全性和协作流畅度。

3 云原生产品家族-开发者服务

微服务引擎

微服务引擎（Micro Service Engine）是面向业界主流开源微服务生态的一站式微服务治理平台，兼容Spring Cloud、Dubbo微服务框架，提供高可用、免运维的服务注册中心（支持Eureka/Nacos/ZooKeeper）、配置中心（支持Apollo）和监控中心（支持Skywalking），实现对微服务的治理和监控。



应用场景

SpringCloud的服务接入和治理：Spring Cloud提供了一系列微服务工具包，为开发者提供了分布式系统的服务发现、配置管理、负载均衡、消息队列、数据缓存、断路器等功能。同时，大量企业和开源社区为Spring Cloud贡献了很多实用的功能与组件，完善了Spring Cloud生态圈。

Dubbo的服务接入和治理：Dubbo是一款高性能、轻量级的开源Java服务框架，它提供了以下核心功能：面向接口代理的高性能RPC调用、智能容错和负载均衡、服务自动注册与发现、运行期流量调度。

3 云原生产品家族-开发者服务

企业应用开发平台

一款面向业务开发者的低代码企业应用搭建平台，支持自定义搭建表单、流程等，满足企业多种场景需求，大幅度提升应用开发效率。



应用场景

企业/政府/销售等领域：提供多行业的运营解决方案，支持用户自主搭建或直接安装相关应用，简化企业多端办公场景，统一流程、应用、信息、文档、看板、搜索入口。灵活定制，提高团队工作效能。

开发服务商：基于PaaS能力为开发服务商提供应用搭建平台，快速实现企业应用孵化。同时赋能ISV，上传标准应用至应用市场，丰富企业应用生态体系。

3 云原生产品家族-数据库

云数据库MySQL

云数据库MySQL是一款开箱即用、性能卓越、安全可靠的在线数据库服务，实例秒级开通、性能深度调优、安全企业级标准。产品支持MySQL数据库主流版本引擎，提供异地高可用、弹性伸缩、智能运维等云原生高级特性，满足全行业数据库需求。



产品优势

便宜易用: 即开即用、无缝兼容、运维便捷

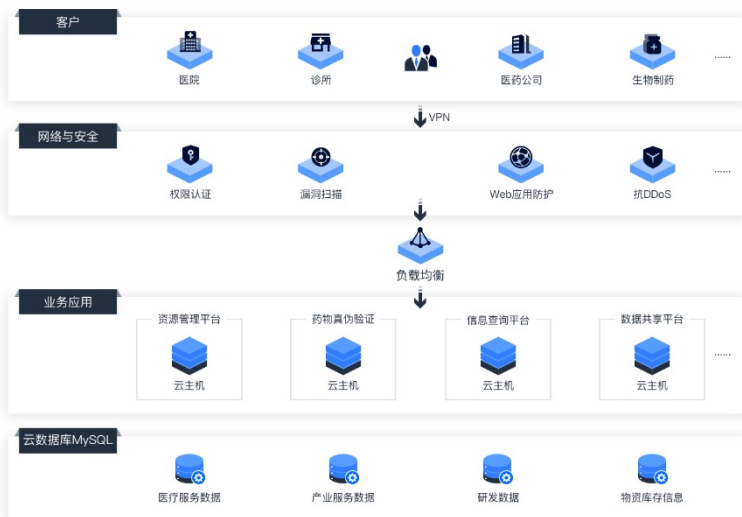
高性能: 参数与SQL优化、可靠的IaaS基础设施、高速访问

安全可靠: 敏感信息传输加密、数据存储加密

3 云原生产品家族-数据库

应用场景

- **电子政务：**电子政务服务每天面临数十亿人次的大规模访问，云数据库提供高吞吐、高并发、安全可靠的数据库服务。
- **医疗医药：**医疗医药需要对数据实现实时分类存储、管理与查询。云数据库为您提供安全、稳定、可靠的数据库服务。



- **金融领域：**金融服务每天存在大规模高频访问交易量。云数据库为您提供安全稳定、高性能、数据强一致的数据库服务。
- **电子商务：**云数据库为您提供读写分离架构的高性能架构，配合云数据库Redis 快速读写能力，助力访问高峰带来的请求压力。

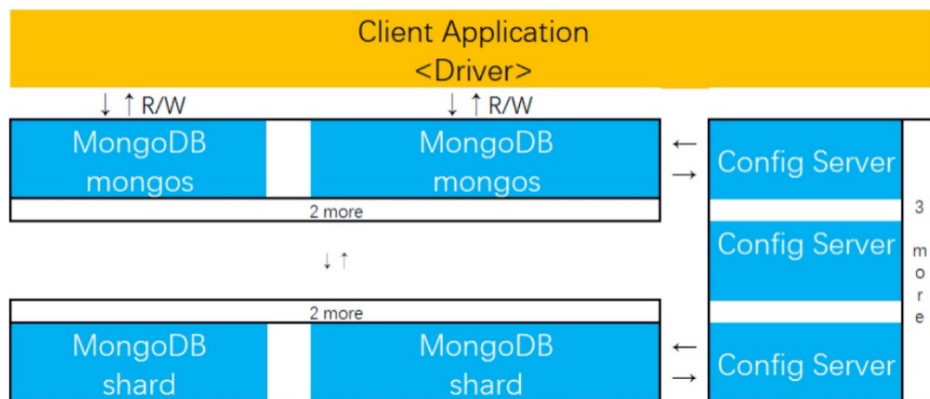


3 云原生产品家族-数据库

文档数据库

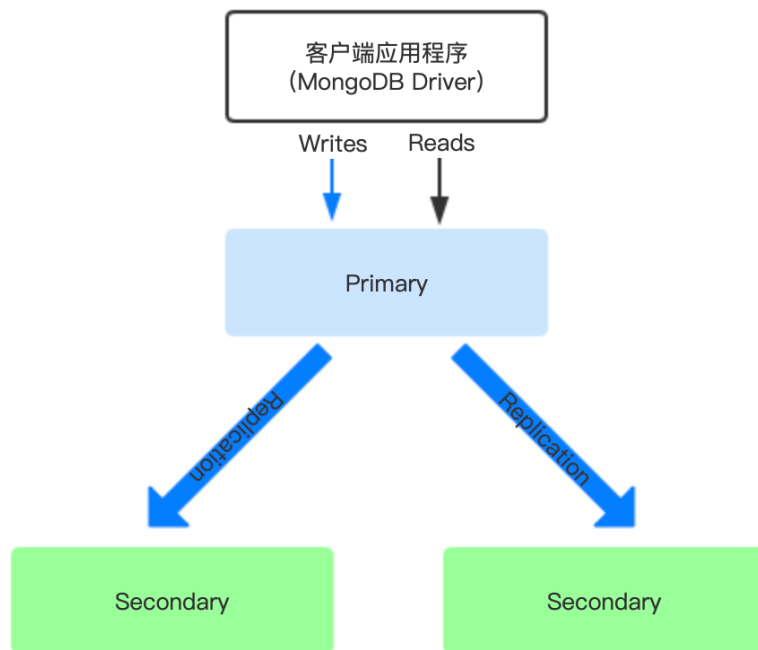
移动云文档数据库服务是一款完全兼容MongoDB协议的NoSQL数据库，提供安全易用、稳定可靠、弹性伸缩能力，满足互联网、电商、游戏、直播、物联网场景应用。

分片集群



- Mongos节点：单节点架构，负责将查询和写操作路由到分片集群实例的对应分片中。
- Shard节点：每一个Shard就是一个副本集，负责存储数据。
- ConfigServer节点：三节点架构，负责存储集群和分片的元数据。

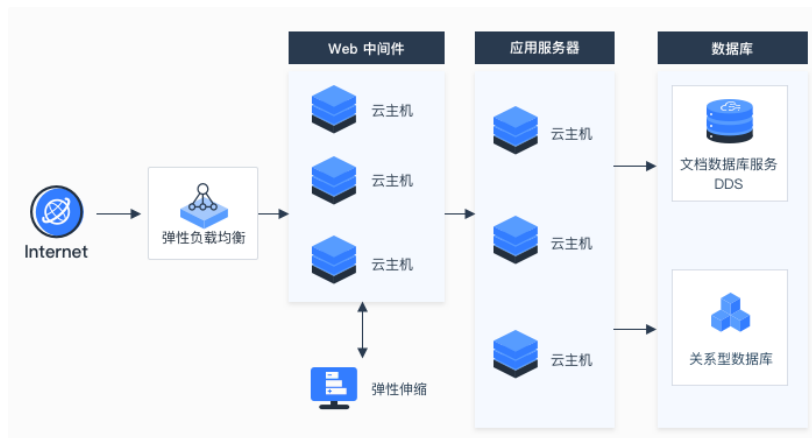
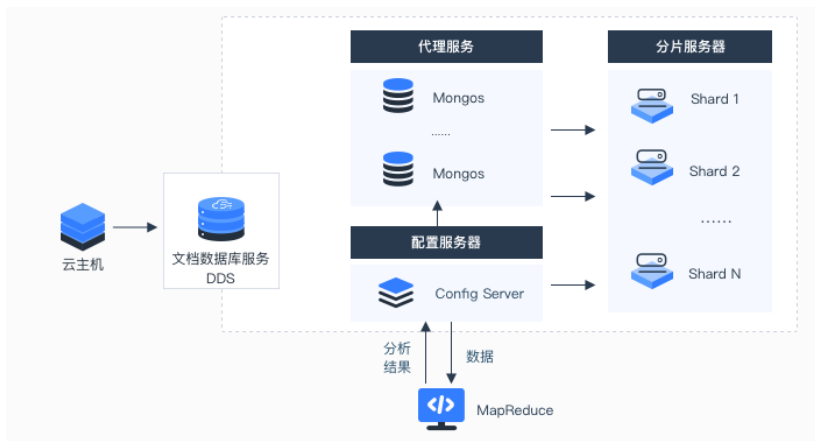
副本集群



- Primary节点：负责读写操作的节点。每个副本集实例中只能有一个Primary节点。
- Secondary节点：通过oplog（操作日志）同步Primary节点的数据，可在Primary节点故障时通过选举成为新的Primary节点，保障高可用。
- ReadOnly节点：只读节点，可应用于没有写请求，有大量读请求的场景，以释放Primary和Secondary节点的访问压力。

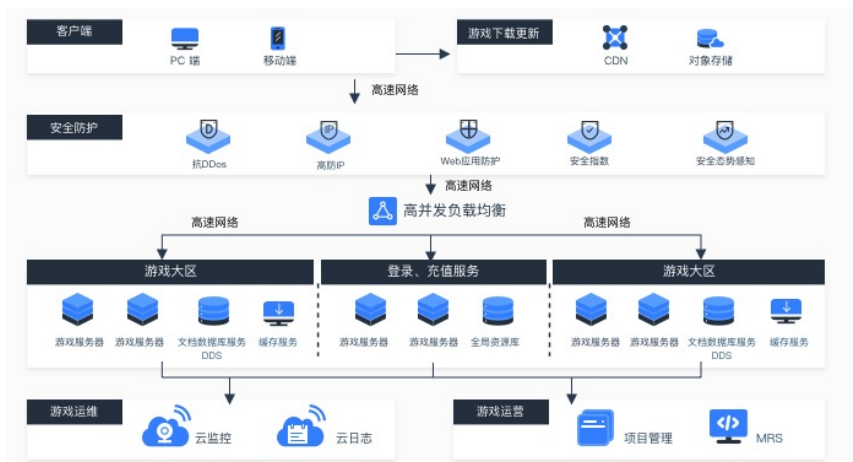
3 云原生产品家族-数据库

应用场景



- **位置应用：**许多移动互联网应用，如网约车、社交APP等，需要采集用户地理位置信息，存储多样的数据类型，并对这些复杂数据进行查询和分析。

- **大数据：**新的数据业务场景下，通常需要将来自不同领域以及同一类型的海量数据，按照设定的分析逻辑处理后形成新的数据，并将这些数据分解、组合、转换之后，应用于新的业务场景。
- **游戏：**在游戏应用中，可以将大量的用户信息存储在文档数据库中。游戏玩家活跃高峰期可采用分布式集群架构，通过Mongos和Shard水平扩展，应对高并发场景。

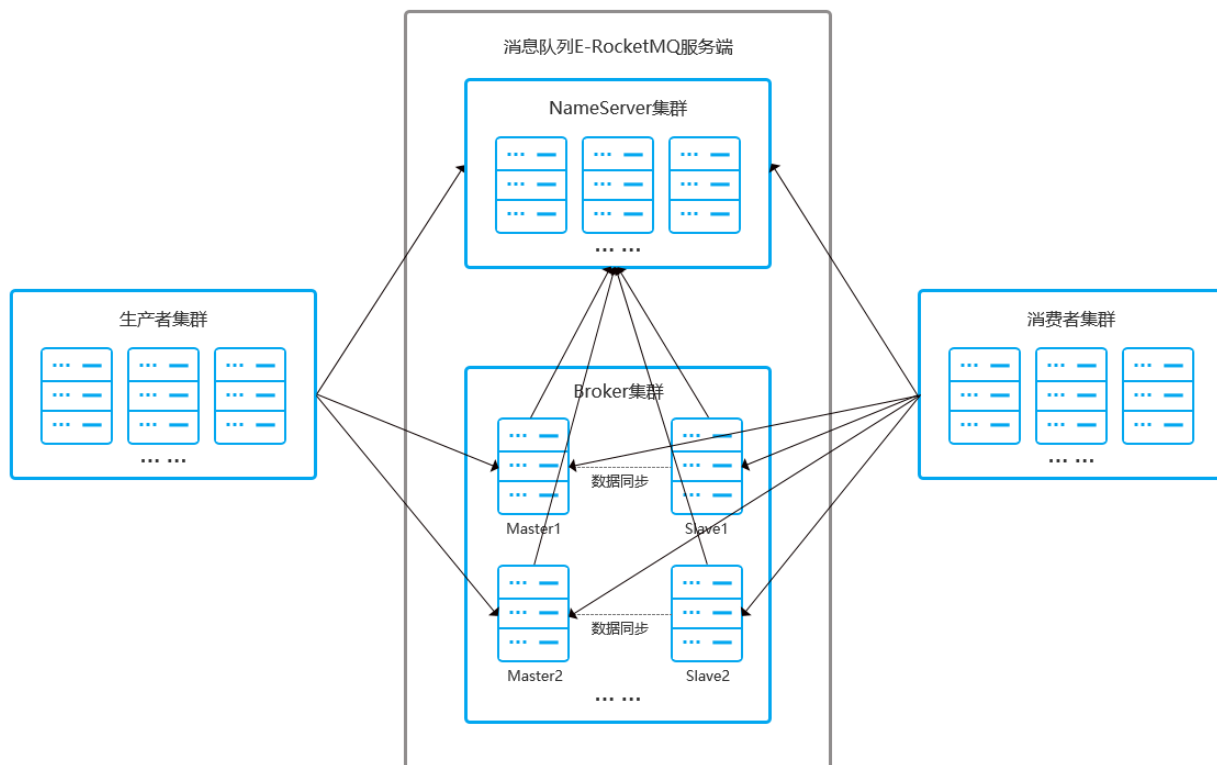


3 云原生产品家族-消息队列

消息队列 RocketMQ

基于分布式集群技术，为用户提供安全可靠、简单易用、低延迟、高可用的消息队列服务，可用于分布式部署的不同应用间消息存储与收发，广泛应用于系统异步解耦、流量削峰等业务场景，满足电商、金融、政企等多种行业使用需求。

系统部署架构



•NameServer

是一个几乎无状态节点，可集群部署，在消息队列RocketMQ中提供命名服务，更新和发现Broker服务。

•Broker

消息中转角色，负责存储消息，转发消息。

生产者

与NameServer集群中的其中一个节点（随机）建立长链接（Keep-alive），定期从NameServer读取Topic路由信息，并向提供Topic服务的Master Broker建立长链接，且定时向Master Broker发送心跳。

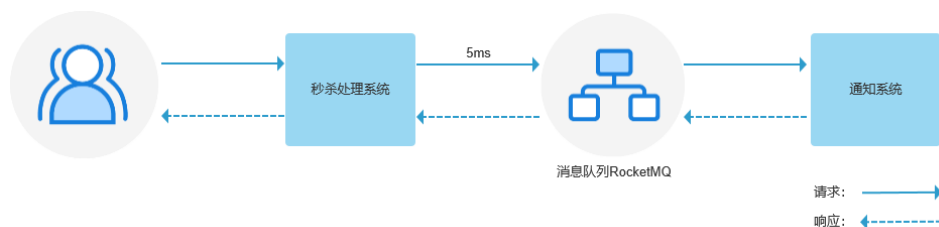
•消费者

与NameServer集群中的其中一个节点（随机）建立长连接，定期从NameServer拉取Topic路由信息，并向提供Topic服务的Master Broker、Slave Broker建立长连接，且定时向Master Broker、Slave Broker发送心跳。Consumer既可以从Master Broker订阅消息，也可以从Slave Broker订阅消息，订阅规则由Broker配置决定。

3 云原生产品家族-消息队列

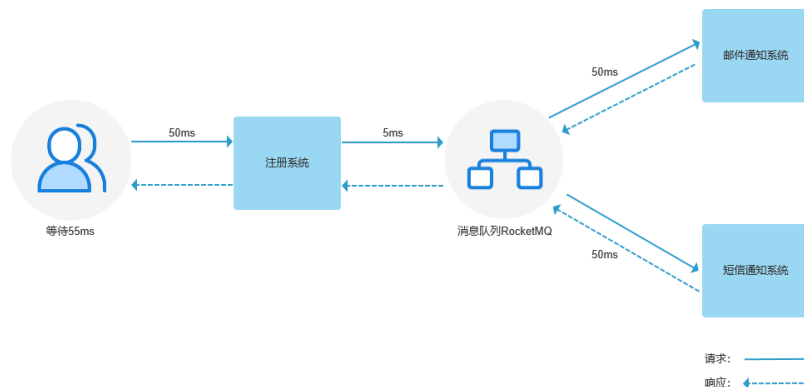
应用场景

- **消息的顺序收发：**消息队列RocketMQ顺序消息为全局顺序，对于指定的一个Topic，所有消息将按照严格的先入先出（FIFO）的顺序，进行顺序发布和顺序消费。



- **削峰填谷：**在秒杀或团队抢购活动中，由于用户请求量较大，导致流量暴增，秒杀的应用在处理如此大量的访问流量后，下游的通知系统无法承载海量的调用量，甚至会导致系统崩溃等问题而发生漏通知的情况。为解决这些问题，可在应用和下游通知系统之间加入消息队列RocketMQ。

- **大规模机器的缓存同步：**在某些网页数据实时变化的场景下，缓存技术便无法满足对数据的实时访问需求，多次的查询将会对页面打开速度产生影响。使用消息队列RocketMQ的广播消费模式，那么这条消息会被所有节点消费一次，相当于把相关数据信息同步到需要的每台机器上，取代缓存的作用。



- **异步解耦：**将比较耗时而且不需要即时（同步）返回结果的操作作为消息放入消息队列。同时，由于使用了消息队列RocketMQ，只要保证消息格式不变，消息的发送方和接收方并不需要彼此联系，也不需要受对方的影响，即实现解耦。

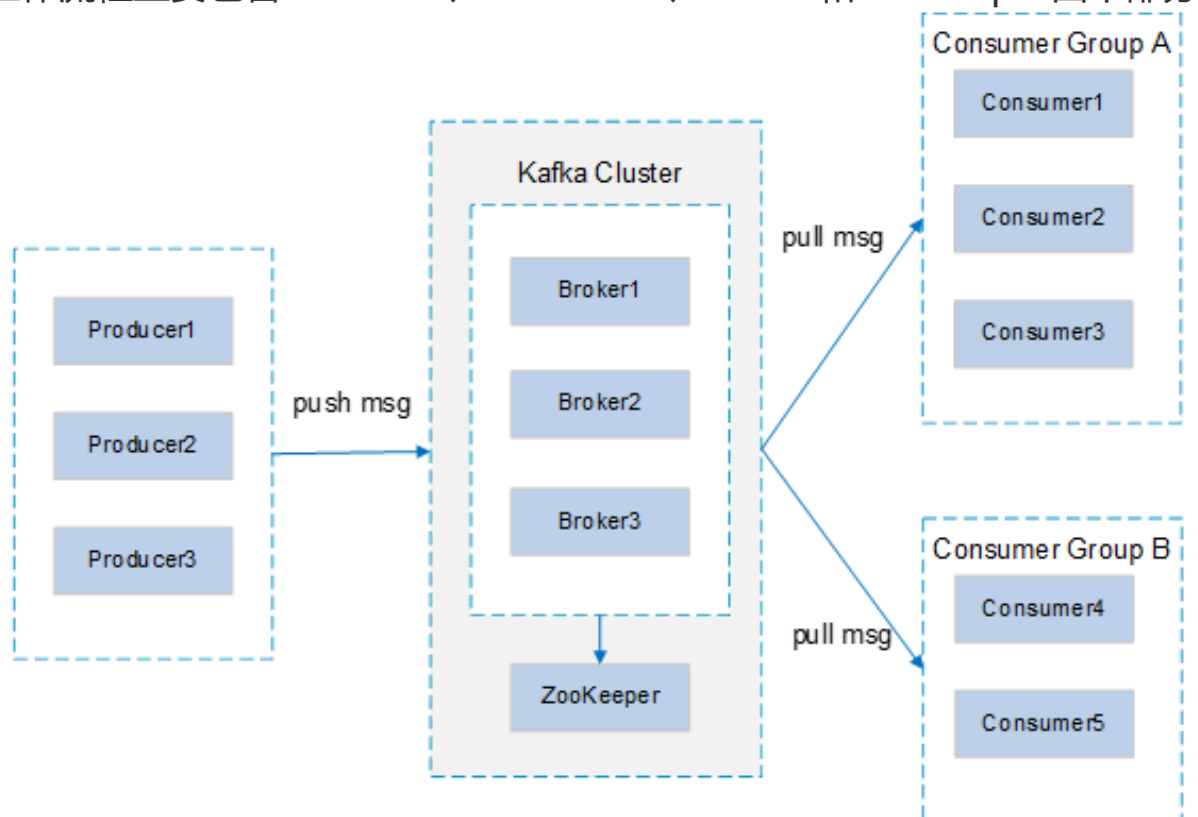
3 云原生产品家族-消息队列

消息队列 Kafka

移动云消息队列Kafka是一种分布式、高吞吐、高可扩展的消息队列服务，具有高可用、高并发访问、大规模、完全托管的特点，广泛用于日志收集、监控数据聚合、流式数据处理、数据中转等应用场景。

工作流程

Kafka的工作流程主要包含Producer、Consumer、Broker和ZooKeeper四个部分。

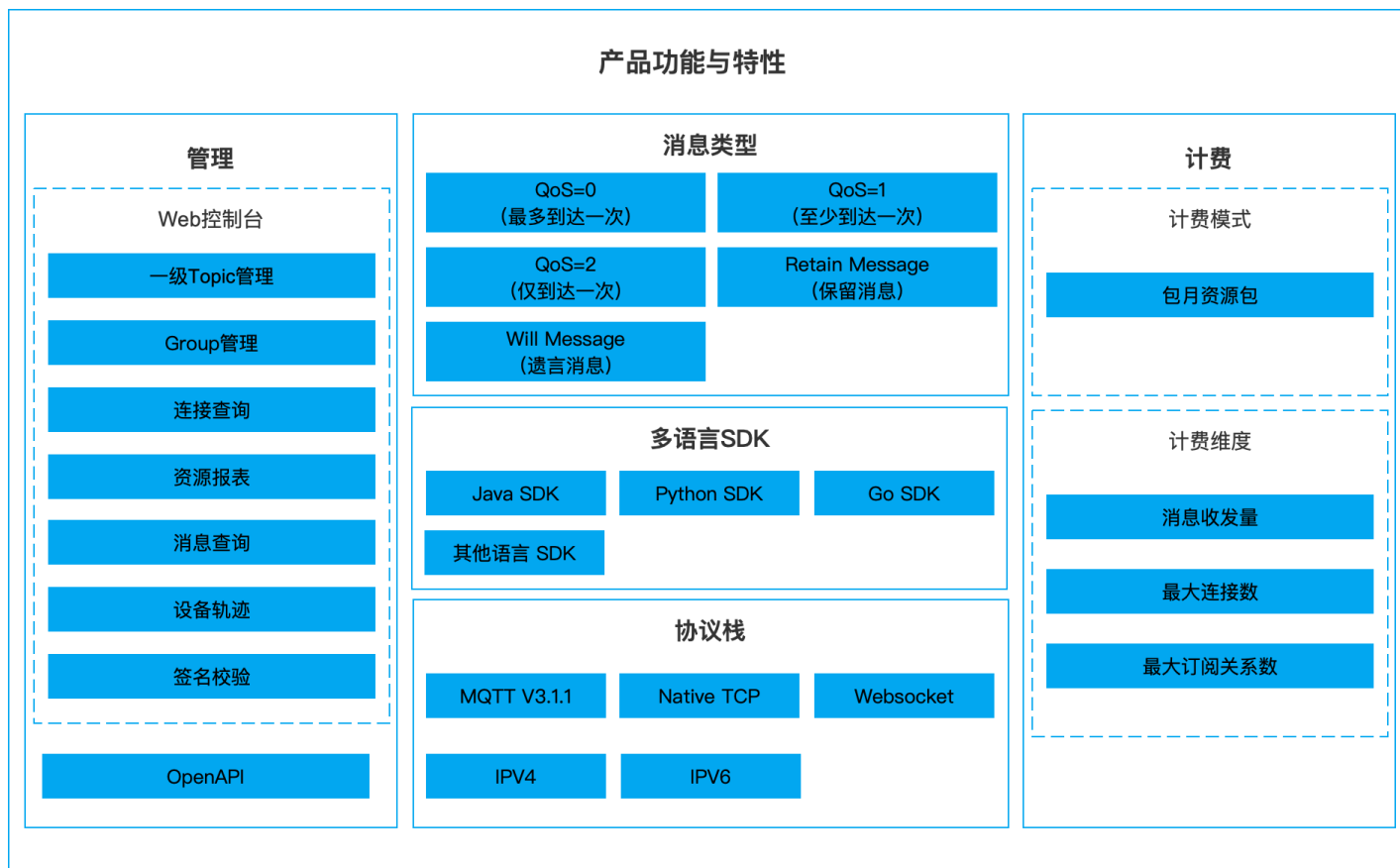


3 云原生产品家族-消息队列

消息队列MQTT

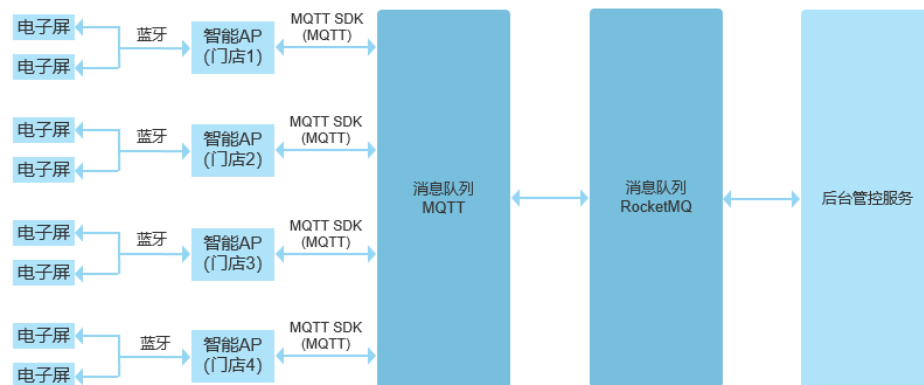
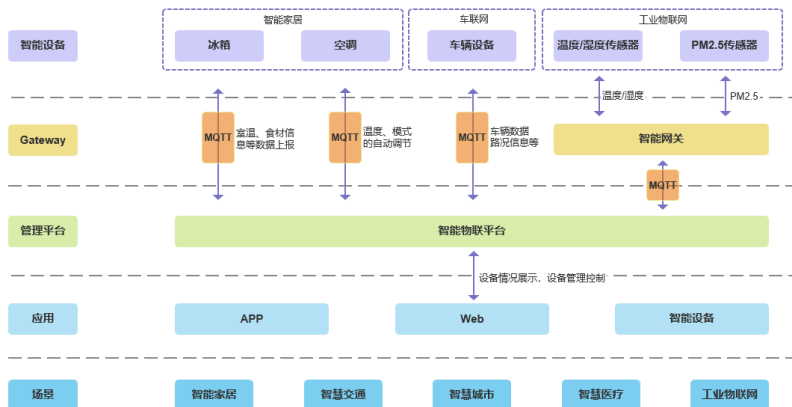
针对移动互联网、物联网等场景设计的轻量级消息中间件产品。支持MQTT V3.1.1协议，支持TCP、Websocket及其加密传输，覆盖绝大多数移动端开发平台及开发语言（JAVA、iOS、Android、nodeJS等）

产品功能视图



3 云原生产品家族-消息队列

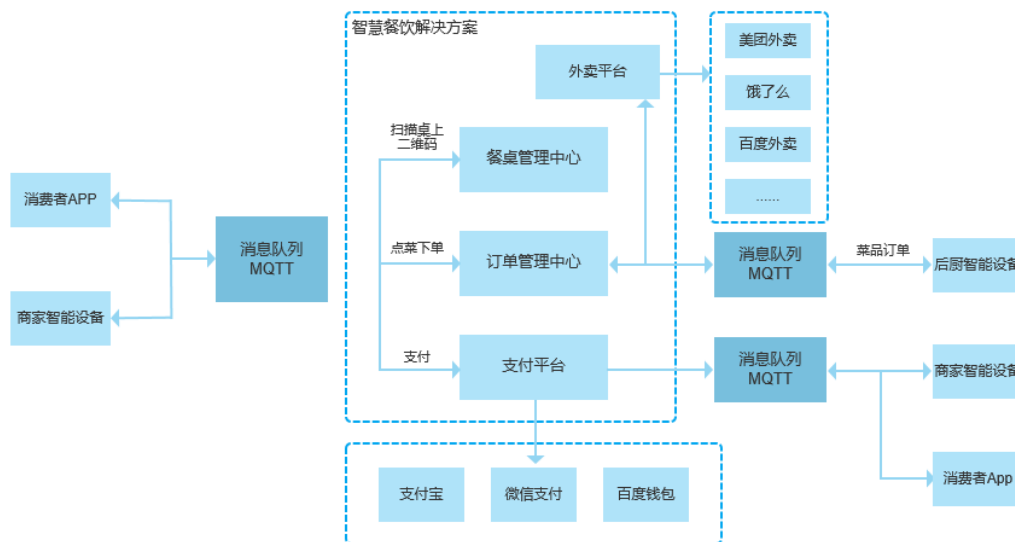
应用场景



- **新零售**：基于消息队列MQTT，可实现商场超市等场所的电子标签、公共场所的多媒体屏幕的显示更新管理。

- **物联网消息**：智能家居、车联网、工业物联网

- **智慧餐饮**：随着物联网行业在各个领域的广泛应用，智能点餐已成为餐厅的标配；通过消息队列MQTT产品，消费者可在餐桌上扫码点餐，并与商家后端系统连接实现自助下单、支付。

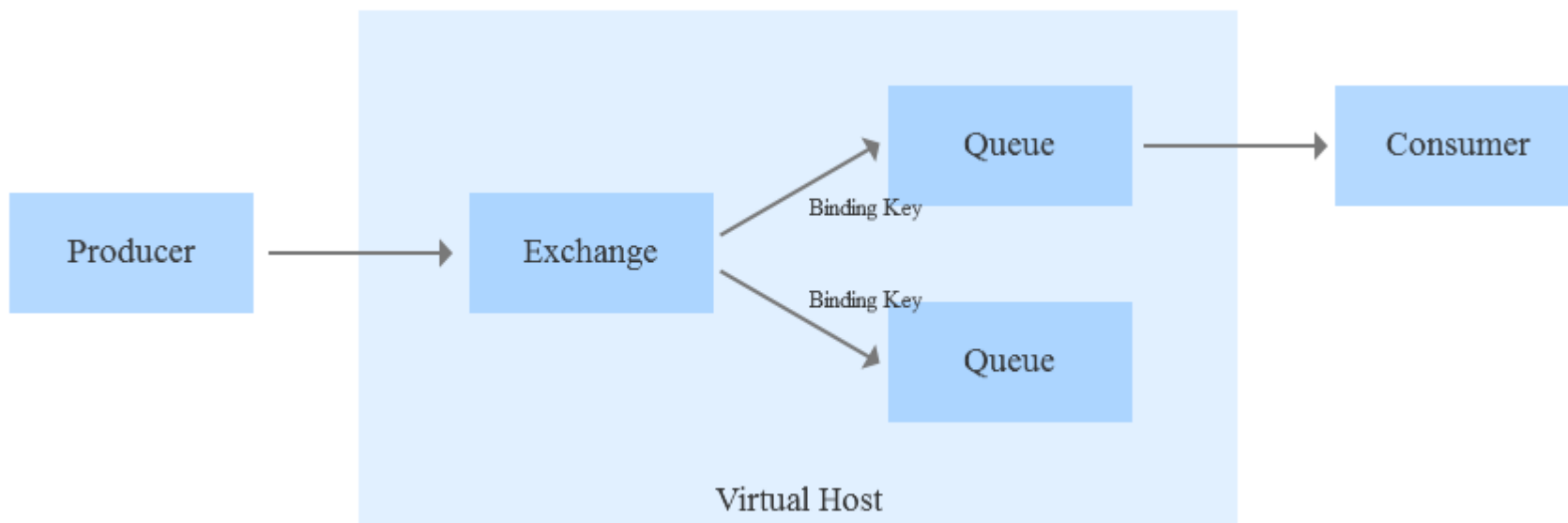


3 云原生产品家族-消息队列

消息队列 RabbitMQ

RabbitMQ 兼容开源RabbitMQ客户端，解决开源各种稳定性痛点（例如消息堆积、脑裂等问题），同时具备高并发、分布式、灵活扩缩容等云消息服务优势，广泛应用于流量削峰、系统解耦等场景。

消息收发模型



发送消息

1. Producer和Broker建立TCP连接;
2. 生产者和Broker建立通道;
3. 生产者通过通道将消息发送给Broker，由Exchange将消息进行转发;
4. Exchange将消息转发至指定的Queue。

接收消息

1. 消费者和Broker建立TCP连接;
2. 消费者和Broker建立通道;
3. 消费者监听指定的Queue;
4. 当有消息到达Queue时，Broker将消息推给消费者;
5. 消费者接收消息。

3 云原生产品家族-大数据

Elasticsearch

一种基于开源Elasticsearch搜索引擎打造的高可用、可伸缩的云端全托管的搜索服务，集成Elasticsearch、Logstash、Kibana等服务。它具备基本的数据存储、查询和数据分析能力，并提供了向量检索、SQL查询、安全管理等增强特性。可满足海量数据检索、日志数据分析、报表自动生成、监控告警等场景需求。

主要组件

Elasticsearch（分布式搜索引擎）、Logstash（数据处理管道）、Kibana（数据可视化工具）、高级特性

产品架构



产品功能

实例管理：支持ES实例、Logstash实例创建和管理，支持节点规格升配、节点数量弹性扩容

安全管理：支持自动/手动备份集群和恢复，通过云监控提供监控告警功能

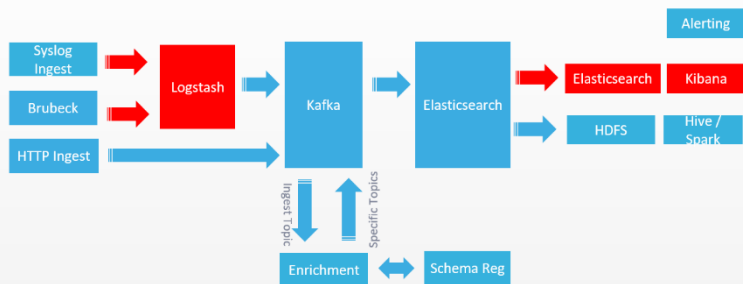
增强特性：支持RBAC的角色管理，索引生命周期管理，支持高性能向量检索、SQL查询等

3 云原生产品家族-大数据

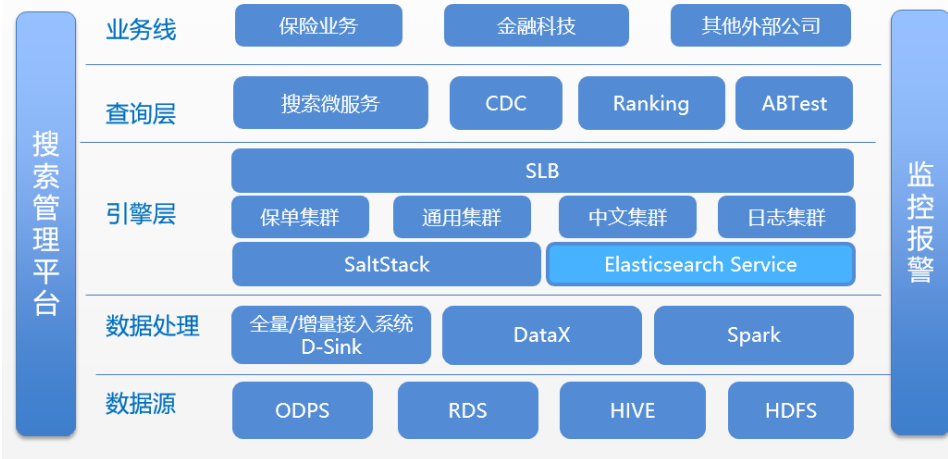
应用场景

企业运维：针对社交、游戏等运营要求高的业务场景，技术团队基于Elasticsearch产品对用户体验进行实时监控，当用户遇到延迟或异常情况时，监控团队可快速定位到异常原因。

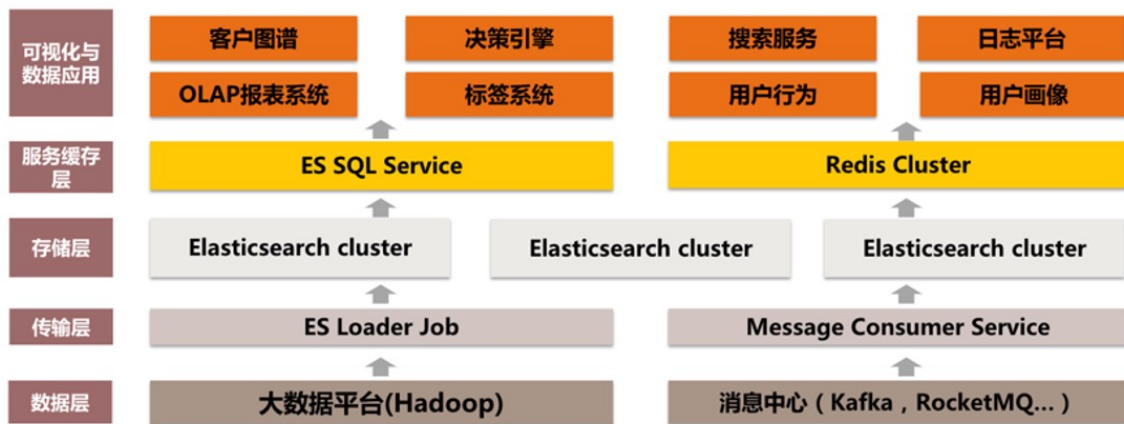
数据流架构



站内搜索：针对保险行业，以互联网生态自我定位，从事电商险，航旅险，寿险，车险，健康医疗险以及金融等业务需要构建搜索引擎的企业客户，存在DB级海量数据，实时性查询难、数据库分库无法查询等问题，基于Elasticsearch为客户构建搜索引擎服务。



商业智能决策：大数据覆盖面广、维度丰富、实时性高和人工智能技术飞速发展的特点，使大数据和人工智能的热点应用领域和方向延伸至企业风控管理体系中。基于Elasticsearch高效的搜索和强大的聚合特性，打造客户主题标签系统，从客户基本维度、业务场景等维度，并结合ES构建的报表服务，实现运营体验升级。



3 云原生产品家族-大数据

云原生大数据分析LakeHouse

云原生大数据分析LakeHouse是提供一站式的流处理、批处理、交互式分析的Serverless融合大数据处理分析服务，企业使用标准SQL即可完成云上多数据源的分析计算，助力企业快速构建数据湖/仓解决方案，洞察业务价值。



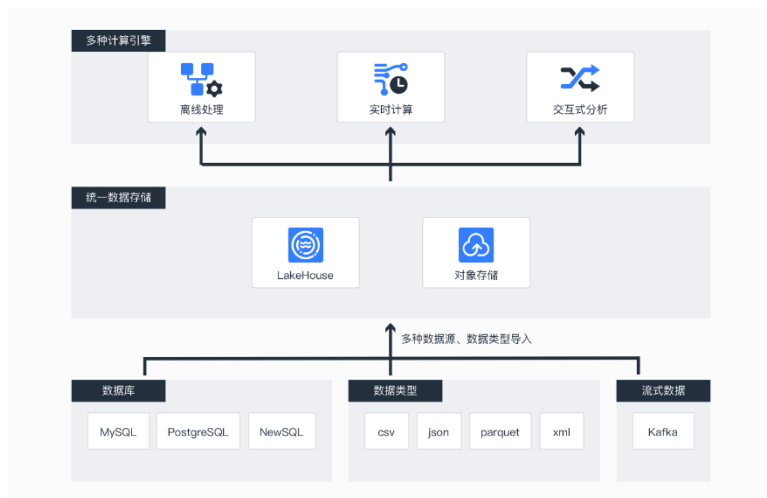
产品优势

云原生弹性：采用计算和存储分离架构，支持计算资源、存储资源按需独立扩缩容；支持按量计费，降低使用成本。

简单易用：通过控制台WEB方式提供服务，服务全托管，开箱即用；产品兼容标准SQL，用户使用SQL即可实现海量数据查询分析，学习使用成本低。

一站式服务：融合移动云数据生态，数据无需搬迁，即可实现对云上多个数据源进行分析；无缝对接数据集成与治理产品，提供一站式数据湖/仓解决方案。

3 云原生产品家族-大数据



应用场景 数据湖构建

在数据量飞速膨胀的趋势下，用户存储资源与计算机资源扩容速度不匹配，存在成本优化方面的诉求；用户数据来源广泛，元数据分散较难管理；用户无法完全预期未来业务的变化，希望存储一份完整的原始数据，后续“按需”贴合业务使用需求。

海量低成本数据分析

直接使用MySQL等业务数据库进行海量数据分析，不仅容易对线上业务产生影响，存储成本高，分析能力也较受限。使用LakeHouse可以通过SQL、Spark等进行多维度分析与处理，同时基于Serverless架构，能够以尽可能低成本满足整体存储和计算的需求。

搭建云原生大数据平台

LakeHouse支持多样化数据来源，包括但不限于应用自身产生的数据、采集的各类日志数据、数据库中抽取的各类数据，并提供离线批处理、实时计算、交互式查询等能力，节省了搭建传统大数据平台需投入的大量软硬件资源、研发成本及运维成本。

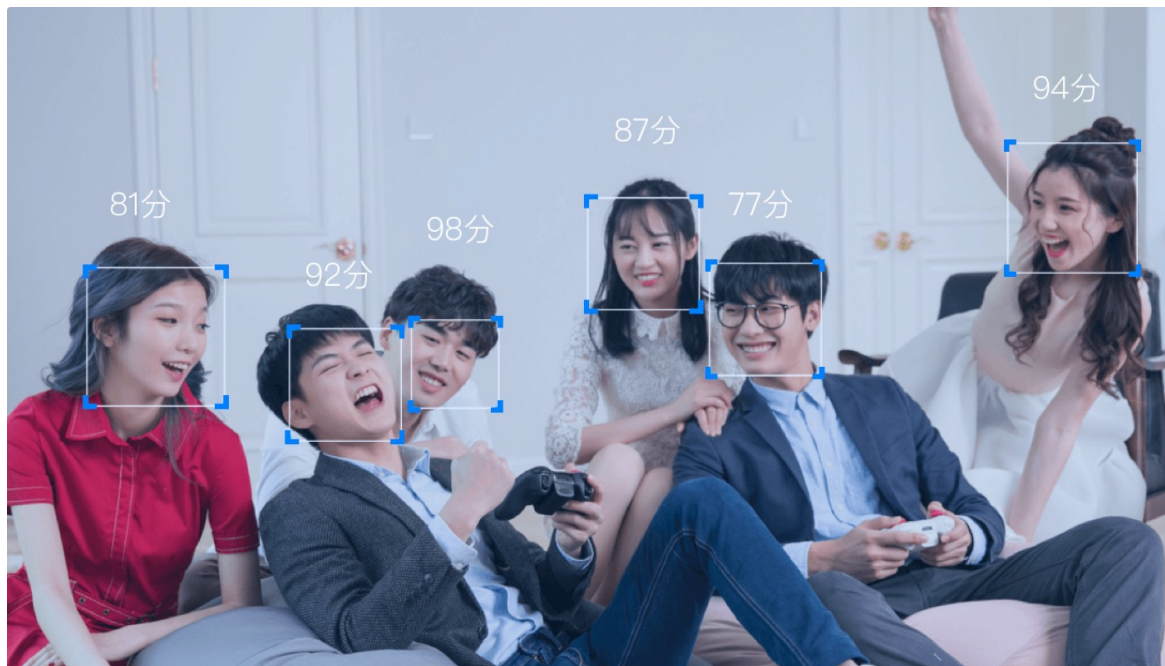
3 云原生产品家族-人工智能

人脸与人体识别

通过分析给定的包含人物面部或人体躯干的图片，实现对图片中人脸及人体的特征数据进行提取并输出特定的结构化数据的功能。

产品优势

功能丰富、稳定可靠、方便高效、简单易用



应用场景

人脸检测：人脸修饰、营销互动、相册分类

人脸搜索：门禁系统、签到考勤、安防监控

人脸检测与属性识别：线下广告投放、智慧零售、营运车辆驾驶监测

活体检测：平台实名认证、金融远程开户

3 云原生产品家族-人工智能

自然语言处理基础服务

基于人工智能技术，针对各类企业及开发者提供的用于文本分析及挖掘的云服务，旨在帮助用户高效地处理文本。

产品优势

海量数据建模：基于海量文本数据进行分析建模，模型精度及适配性高。

精准高效：多种算法模型集成学习，短时间内分析处理海量文本，理解深层次文本语义信息，更精确地挖掘出文本价值。

快速迭代：深度应用于移智舆情通产品中，持续快速迭代情感分析模型，紧跟语义时代变化。

简单易用：基于业界领先的深度学习框架，支持分布式GPU卡训练模型标准化接口封装，简单易用，大大降低开发人力成本。

服务稳定：依托移动云强大的技术实力，提供高质量的自然语言处理服务。

应用场景



- **智能分词：**智能问答、智能搜索



- **情感分析：**舆情监控、口碑分析、满意度分析



- **关键词提取：**智能问答、智能搜索、内容推荐

3 云原生产品家族-人工智能



- **文本信息抽取：**文档整理、知识库构建、智慧审计



- **文本分类：**个性化推荐、大规模文本检索



- **情绪识别：**智能客服助手、服务质量评估



- **命名实体识别：**简历自动汇总、客户支持简化

4 应用场景

企业级云原生数据服务场景

基于云原生开源Kubernetes Operator深度集成消息队列、数据库、人工智能、大数据等一站式全链路数据管理与服务

面向对象

面向企业已逐步适应IaaS能力，需要进一步聚焦业务及创新技术，构建PaaS服务的场景



优势

- **统一管理，资源集约：**通过中间件服务化和平台化提供、解决各类中间件分散提供和管理带来的资源大量占用的浪费问题
- **标准接入，高效稳定：**通过表转化方式逐步积累对各类中间件的积累深度，提升应用整体的稳定性
- **组件下沉，聚焦业务：**将支撑应用的数据服务模块化、系统化、并下沉至云原生技术平台，支撑企业将更多的精力聚焦于业务创新

4 应用场景

迁移上云场景

高效整合移动云基础设施及PAAS层的产品能力，为客户应用提供一站式DevOps应用云上交付，充分利用云的弹性和资源服务化优势，构建灵活高效、稳定可靠、资源集约的应用系统

面向对象

面向企业快速上云、DevOps的场景，助力企业快速实现云原生转型



优势

- **标准流程：**一站式容器化交付，覆盖应用规划、需求、设计、开发、测试、交付、运维完整生命周期
- **灵活高效：**全容器化架构设计，任务调度更灵活，执行效率更高
- **稳定可靠、资源集约：**可以根据业务流量自动对业务扩容/缩容，避免流量激增扩容不及时导致系统挂掉，以及平时大量闲置资源造成浪费

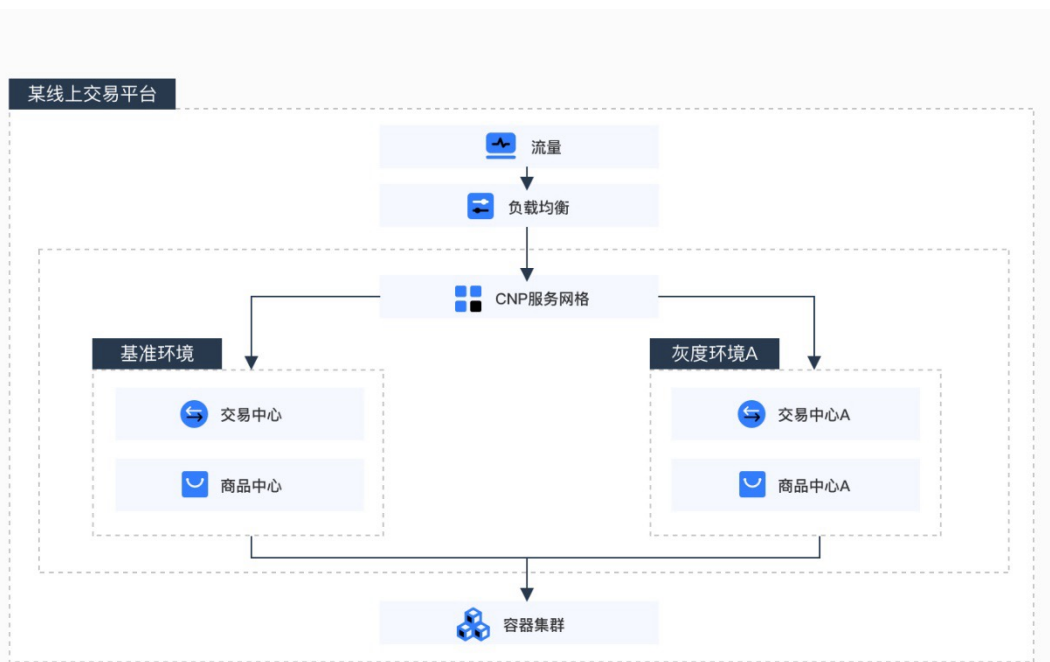
4 应用场景

服务治理场景

集成开箱即用的Istio服务流量治理能力，用户无需修改代码，即可实现灰度发布、流量治理和流量监控能力

面向对象

面向企业深度用云的场景，开发者可以更聚焦于业务逻辑本身，提升应用开发效率并加速业务探索和创新



优势

- **组件下沉，聚焦业务：**将服务的网络通信作为基础设施下沉，作为网格统一管理
- **使用便捷、易于迭代：**将 SDK 拆解为单独进程，并以 Sidecar 模式进行部署，将服务治理功能与业务逻辑剥离，使得业务进程专注于业务逻辑
- **无侵入接入、治理全面：**业务代码无侵入兼容istio微服务框架，提供负载均衡、流量加密、熔断限流、故障注入、灰度发布等全方位服务治理功能

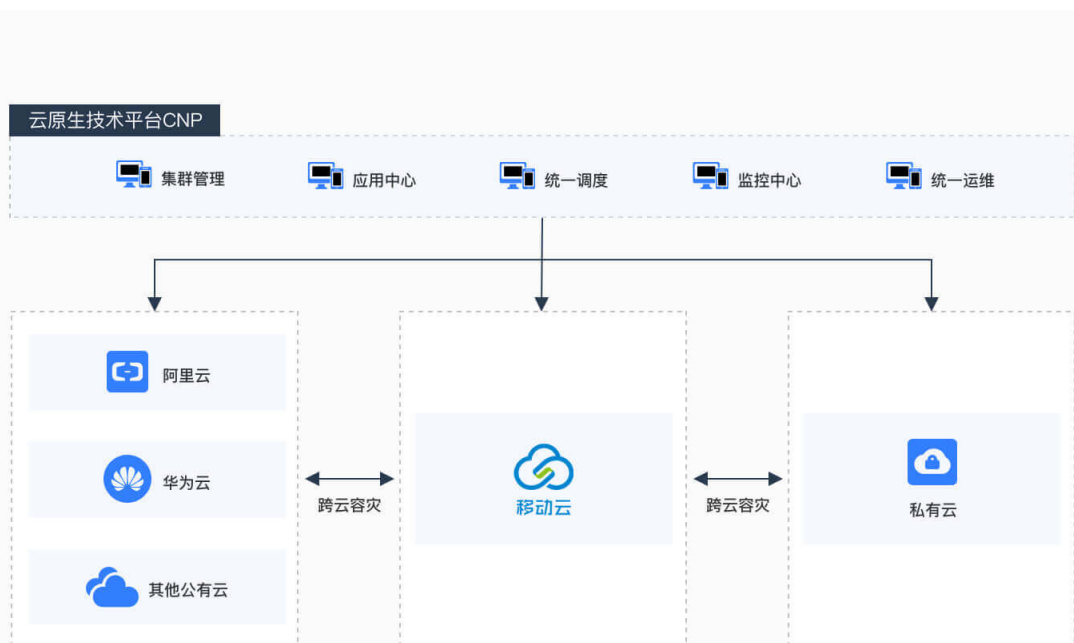
4 应用场景

多云混合编排场景

基于CNCF社区Karmada项目，在公有云、私有云和混合云场景下，提供可插拔，全自动化管理多集群应用，实现多云集中管理、高可用性、故障恢复、流量调度及容灾

面向对象

面向企业具有多家云服务商资源，需要做跨云容灾、跨云调度的场景



优势

- **统一管理**：提供公有云、私有云、混合云等多集群统一跨云容灾及调度
- **高扩展性**：比单集群更高的横向扩展性，解决单集群管理容量受限的问题
- **降低成本**：跨云业务的构建和维护较为简单，无需关注大量基础设施的问题
- **厂商解绑**：多云混合编排，避免供应商锁定，增强用户的商业灵活性

5 解决方案-政务云原生解决方案

政务云原生解决方案

移动云打造全面以应用为中心的算网原生新型政务云，围绕云原生理念及治理标准，实现政务业务应用从开发、集成，到监控、运维的统一管理，满足应用级可视可管、精细化资源运营、标准化可共享的需求。

方案架构



方案优势

先进的云原生理念：面向算网原生设计，提前规划算力调度，支持两级调度，算力资源利用率提升。

丰富的数据服务：提供数据库、大数据、中间件、AI等丰富的云原生数据服务，旨在降低云原生技术的使用门槛，帮助政务系统更快捷的构建云上业务。

完备的灾备体系：提供完备的省市两级容灾双活方案，无缝实现政府用户业务的备份和容灾，支持业务流量的智能分流，实现政务应用的一键式容灾双活。

一站式运维服务：提供实时展示资源链路追踪，运维监控，全局统一视图等全面的云原生可观测能力，助力企业用户一站式运维。

5 解决方案-政务云原生解决方案

客户案例



中国移动承建某省数字政府多个政务云项目，遵从“政务一朵云”的整体架构和建设思路，基于移动云全栈云原生能力，成功通过ECSC-CNP完成多个云平台、技术栈接入管理。多种异构云平台，包括移动、华为、华三、浪潮、VMware等多个技术栈，实现全省各级政务云的集中管理和综合服务。

5 解决方案-医疗云原生解决方案

医疗云原生解决方案

移动云医疗云原生解决方案基于云原生全栈产品及服务面向私有云、专属云、公有云等多场景，为医疗行业客户提供高效敏捷、灵活弹性、业务智能，安全可信的新型医疗信息化服务。

方案架构



方案优势

多云混合编排：支持多云集群统一管理、多云灾备等，使医疗行业在不改变原有企业IT架构及应用系统、不需要迁移数据的情况下，享受到云原生的优质服务。

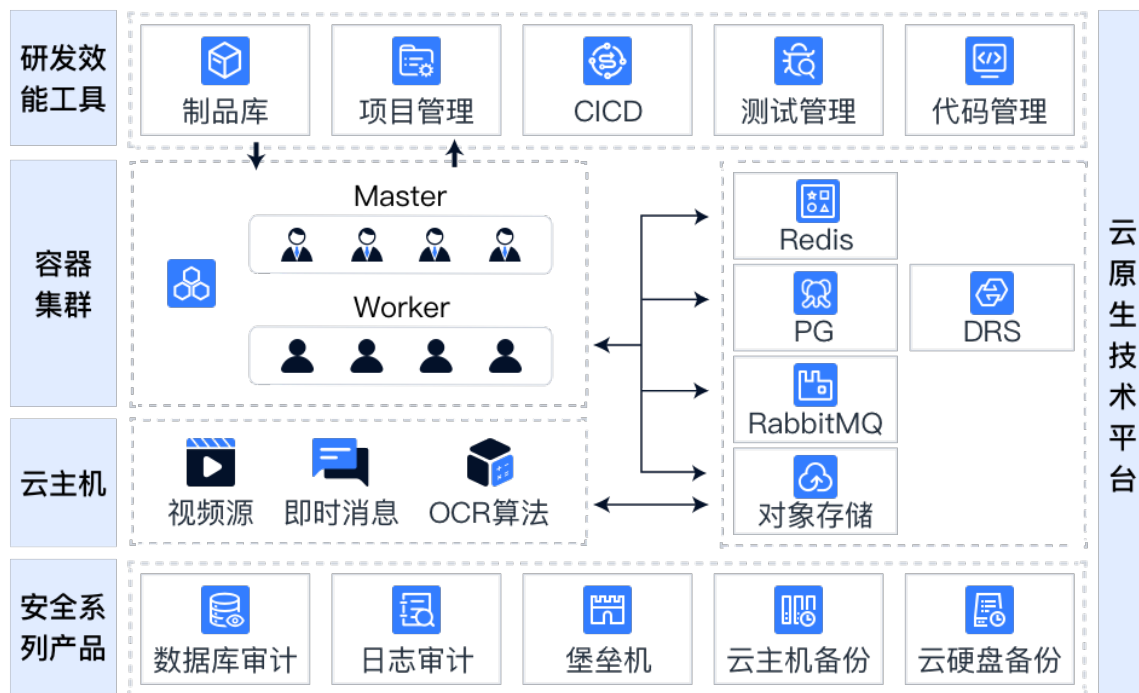
基于云原生的智能化：云原生数据平台及AI机器学习能力帮助医疗行业打造智能化业务体系，助力智慧医疗、智慧服务、智慧管理等。

全生命周期管理：DevOps敏捷开发平台跨越软件生命周期的需求、设计、开发、测试、运维部署、运营反馈等环节，提升综合管控的便利性。

完整的服务治理能力：支持服务注册/发现、智能路由、熔断/限流、微服务应用拓扑展示、配置管理、性能监控等服务治理功能，运维管理更加便利。

5 解决方案-医疗云原生解决方案

客户案例



艾昆纬基因库上云项目，通过移动云全栈云原生产品及服务，为受试者提供视频与研究进行远程访视、即时通讯、家庭护理相关管理及移动端数据采集等功能。不仅助力艾昆纬提高资源利用率，加速资源整合和优化，推动企业数字化、智能化转型，也为后续业务拓展带来了极大便利。



问题

1. 上云和云原生的区别是什么？
2. 单体应用一定落后吗？微服务一定先进吗？
3. 为什么容器适合云原生？
4. 为什么无状态对扩容很重要？
5. 服务网格解决了什么问题？
6. 多云混合是不是越多云越好？

谢 谢



南京大學
NANJING UNIVERSITY